Segger: fast and accurate cell segmentation of imaging-based spatial transcriptomics data

1 Methods

1.1 Overview

Segger is a graph neural network (GNN)-based method that formulates cell segmentation in imaging-based spatial transcriptomics (iST) as a problem of transcript–cell assignment. Rather than relying solely on geometric proximity, Segger assumes that a transcript belongs to a cell if it is consistent both with its local spatial context and with the gene expression programs that define cell identity, as captured in a shared latent representation. Segger encodes iST data as a heterogeneous graph with transcript and cell nodes, connected by spatial transcript–transcript edges linking physically proximate transcripts and transcript–cell edges derived from the initial (typically nuclear) segmentation. A graph attention network propagates information across this structure to learn a joint embedding of transcripts and cells, in which correct associations correspond to proximity in the learned space. The model is trained on high-confidence assignments from the initial segmentation and subsequently refines these assignments during inference by removing inconsistent transcripts and linking previously unassigned ones to their most likely cells. The resulting segmentation is defined by the set of transcripts associated with each cell (**Fig. 1**).

A central challenge in iST data is that transcript assignment is inherently noisy. Even with accurate segmentation boundaries, transcripts assigned to a cell may originate from neighboring cells due to molecular diffusion, optical crowding, and the projection of three-dimensional tissue into two dimensions (**Fig. 1a**). As a result, contamination is not merely a consequence of imperfect boundary delineation, but an intrinsic property of the data. Notably, this effect is already evident in nucleus-based segmentation, where transcripts from distinct cell types can be observed within the same nuclear region (**Fig. 1b**), indicating that even conservative assignment strategies are not free of contamination. While cell boundary staining can in principle recover a larger fraction of cytoplasmic transcripts, it is often variable across tissues and experimental conditions, and the resulting segmentations are less stable than nuclear boundaries. Nuclear staining, by contrast, provides a consistent and high-confidence prior across platforms, though it captures only a subset of the transcriptome. Segger therefore uses nuclear segmentation as a stable starting point and resolves missing and ambiguous assignments using transcriptional consistency in the learned latent space, within a staining-derived candidate field, by default the slightly buffered cell segmentation (Section 1.5.1).

To construct the shared embedding space that enables transcription-aware assignment, Segger employs three coupled loss functions (**Extended Data Fig. 1**). A cell embedding loss anchors cell nodes to the phenotypic manifold defined by their gene expression profiles, so that cells of similar transcriptional state occupy nearby positions in the latent space. A transcript embedding loss organizes transcript nodes by gene-gene co-expression structure. A segmentation loss then operates within this already-organized space to refine transcript–cell assignments: Because cell and transcript embeddings are established first, the segmentation objective performs precise contrastive refinement — pulling transcript–cell pairs that are consistent with both spatial proximity and transcriptional identity together, and pushing inconsistent pairings apart — rather than having to discover this structure from scratch. The three losses are applied in a staged manner using cosine scheduling, such that cell and transcript embeddings are established first before emphasizing the segmentation objective. To ensure locality, assignments are further constrained by a spatial receptive field that limits candidate cells for each transcript to its physical neighborhood.

Segger takes as input transcript coordinates, gene identities, initial per-transcript cell assignments, and cell and nucleus boundary polygons, and constructs a heterogeneous graph that is processed by a graph attention network (GATv2). At inference, transcript–cell assignments are determined by cosine similarity in the learned embedding space, within a spatial receptive field that constrains each transcript to candidate cells in its physical vicinity, so that a transcript can be retained in its initial cell, reassigned to a neighbouring cell, removed, or newly recovered. The method scales to datasets exceeding 10^9 transcripts across Xenium, MERSCOPE, CosMx and Atera platforms through an adaptive spatial tiling strategy (**Extended Data Fig. 2**). Guidelines on how to set model parameters are discussed in Section 1.7.

Core model, optimization and graph-scale settings are fixed across datasets; no parameter was retuned by dataset (Section 1.7).

1.2 Heterogeneous graph construction

Segger represents an iST dataset as a heterogeneous graph in which the nodes encode the two biological entities relevant to segmentation — transcripts and cells — and the edges encode the relationships between them. Transcript nodes encode spatial coordinates and gene identity; cell nodes encode expression-derived embeddings derived from the initial nuclear segmentation. Two types of edges connect them: transcript–transcript edges that capture local spatial co-localization among transcripts, and transcript–cell edges that encode candidate assignments. Graph construction proceeds from raw platform output — decoded transcripts with coordinates and gene labels, and nucleus boundary polygons — and produces a graph that is the sole input to the GNN. The following sections describe how nodes, edges, and their associated features are constructed.

1.2.1 Inputs

As input, Segger accepts iST data consisting of detected transcripts with X-Y-Z coordinates and gene identifiers (the z coordinate, where available, is carried through to the outputs but not used by the model), and 2D polygon segmentation masks of nucleus or cell boundaries derived from fluorescence staining. By default, Segger uses nucleus boundaries derived from DAPI staining as the initial segmentation from which positive training examples are drawn, which provides a stable and high-confidence prior across tissue types and platforms. Although Segger supports cell boundary polygons as an alternative starting point, nuclear segmentation is recommended: cell membrane staining is frequently variable in quality across tissues and experimental conditions, and the resulting boundary segmentations are less stable than nuclear boundaries, even when high-quality membrane staining protocols are applied. Cytoplasmic transcripts not captured by the nuclear mask are recovered during inference through transcriptional consistency in the learned embedding space, within a candidate field defined by the slightly buffered staining-derived cell polygons, rather than by assigning every transcript inside an expanded boundary (Section 1.5.1).

The software supports data from Xenium In Situ (10x Genomics), MERSCOPE (Vizgen), CosMx (NanoString) and Atera (10x Genomics; read with the Xenium reader). For Xenium, transcripts are filtered to quality value (QV) ≥ 20 (QV is a Phred-like score for decoded transcripts; values above 20 correspond to an estimated incorrect-call probability of $\leq 1\%$, the threshold recommended by 10x Genomics). Transcript coordinates are in microns for Xenium and MERSCOPE, and in pixels for CosMx, which are converted to micrometers (0.12028 µm per pixel) before graph construction so that all distance parameters are in physical units. Platform-specific negative-control probes and non-coding control features are removed before further processing. All data is standardized to a common schema comprising transcript coordinates, gene labels, cell IDs, compartment labels (nuclear, cytoplasmic, or extracellular), and boundary polygons with type annotations (nucleus or cell). Each transcript also receives a stable integer row index that links all outputs back to the input molecules. Platform-specific filters, compartment mappings and boundary handling are summarized in **Supplementary Table 1**.

1.2.2 Graph construction

Segger models an iST dataset as a heterogeneous graph $\mathcal{G} = (\mathcal{V}_{tx} \cup \mathcal{V}_{bd}, \mathcal{E}_{tx-tx} \cup \mathcal{E}_{seg})$ containing two types of nodes connected by two types of edges:

- Transcript nodes ($\mathcal{V}_{tx}$), one per detected mRNA molecule, encode a gene identity index and 2D spatial coordinates.
- Cell nodes ($\mathcal{V}_{bd}$), one per staining-derived segmentation boundary, encode an expression-derived embedding vector (Section 1.2.3) and 2D centroid position.
- Transcript–transcript edges ($\mathcal{E}_{tx-tx}$) encode local spatial proximity among transcripts. They are constructed via k-nearest-neighbor (kNN) search using KD-tree in SciPy v1.16 (leaf size 100) with parameters k_{tx} (default 5 points, including the query transcript itself, i.e. at most four neighbouring transcripts and one self-loop, providing sufficient local context without excessive connectivity) and maximum distance d_{tx} (default 5 µm, corresponding to the radius at the lower end of the typical mammalian cell-size range).
- Segmentation edges ($\mathcal{E}_{seg}$) connect each transcript to the cell it overlaps with in the nuclear segmentation (read from the per-transcript input assignment; Section 1.2.4), providing an initial assignment that, despite contamination, contains sufficient signal for the model to learn and subsequently refine.

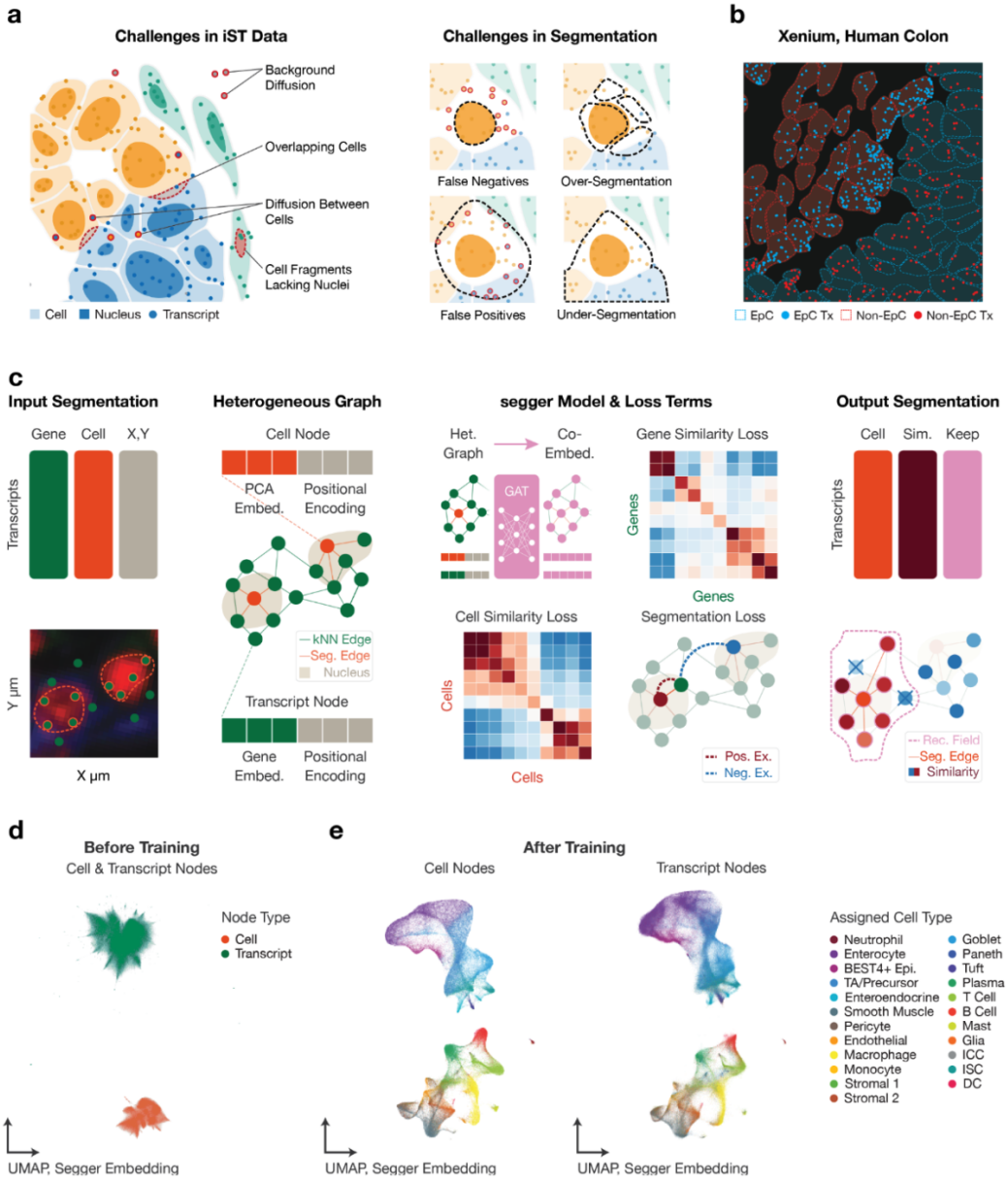


Figure 1 | Segger reframes iST segmentation as transcript-to-cell link prediction. (a) Sources of transcript-to-cell ambiguity in iST (left) produce multiple failure modes (right) when segmentation is treated as a boundary problem. (b) Xenium human colorectal cancer section (Oliveira et al., 2025) with vendor nuclear segmentation, exemplifying common errors in transcript (Tx) assignment to epithelial (EpC) and non-epithelial (Non-EpC) cells. Filtered cells were discarded by the segmentation. Field, 151 × 92 µm. (c) Segger workflow. From left to right: (i) Input segmentation: transcripts are encoded by gene identity and (x, y) coordinates together with an initial cell or nuclear segmentation derived from staining; (ii) Heterogeneous graph: transcripts and cells are represented as distinct node types, with cell nodes represented by fixed (not learned) principal component analysis (PCA) embeddings of their nuclear expression profiles plus positional encoding, transcript nodes by a learned gene embedding plus positional encoding, k-nearest-neighbor edges between transcripts and segmentation edges between transcripts and their initially assigned cells; (iii) Segger model and loss: an attention-based graph neural network (GNN) with a graph attention (GATv2) backbone produces a joint co-embedding of transcript and cell nodes, trained against three losses that act on the receptive field around each cell—a segmentation loss over positive and negative transcript–cell edges, a cell-similarity loss enforcing consistency among cells of the same type, and a gene-similarity loss enforcing consistency among co-expressed genes; (iv) Output segmentation: per-transcript outputs comprising the assigned cell, the model's similarity score and a keep/discard flag, yield a segmentation defined by the transcripts assigned to each cell rather than by a fixed mask. (d,e) Uniform manifold approximation and projection (UMAP) embedding of cells and transcripts in the Segger latent space before (d) and after (e) training, showing that the trained model achieves close alignment of marker transcripts to their respective cells.

Supplementary Table 1: Platform-specific preprocessing applied before graph construction.

Platform	Transcript filters and coordinates	Assignment and compartment mapping	Boundary handling
Xenium (10x Genomics)	Fields `x_location`, `y_location`, optional `z_location`, `feature_name`, `cell_id`, `qv` and `overlaps_nucleus`. Transcripts with `qv` $\geq$ 20 are retained. Features matching `NegControlProbe`, `antisense`, `NegControlCodeword`, `BLANK`, `DeprecatedCodeword` or `UnassignedCodeword` are removed.	`UNASSIGNED`, legacy -1 and string-null tokens denote a missing cell assignment. `overlaps_nucleus` = 1 marks nuclear transcripts; assigned transcripts outside nuclei are cytoplasmic and unassigned transcripts are extracellular.	Flat vertex tables are reconstructed as separate cell and nucleus polygons. `contains_nucleus` marks a cell whose identifier also occurs among the nucleus polygons.
Atera (10x Genomics)	Read with the Xenium reader (same export layout).	As for Xenium.	As for Xenium; the vendor nucleus-anchored cell polygons define the candidate field.
MERSCOPE (Vizgen)	Fields `global_x`, `global_y`, optional z and `gene`. `BLANK` barcodes are removed. No per-transcript quality threshold is applied, as the export has no equivalent of the Xenium `qv`.	A nucleus identifier marks a nuclear transcript and a cell identifier without nucleus identifier a cytoplasmic transcript; transcripts with neither are extracellular.	Annotations that share an entity identifier across z planes are merged into one two-dimensional object; transcripts are assigned to the merged masks with an `intersects` spatial join.
CosMx (NanoString)	Fields `x_global_px`, `y_global_px`, `target` and `CellComp`. Features matching `Negative`, `SystemControl` or `NegPrb` are removed. Pixel coordinates are converted to micrometers.	Nuclear `CellComp` values mark nuclear transcripts, membrane and cytoplasmic values cytoplasmic transcripts, and extracellular or unassigned values extracellular transcripts; null-like cell identifiers are treated as missing.	Cell and compartment rasters are converted to polygons by connected-component extraction and contour tracing. Whole-cell masks comprise all non-extracellular compartments, nuclear masks only the nuclear compartment.
All platforms	Each transcript receives a stable integer `row_index`; all coordinates are in micrometers.	Null assignments are standardized; unassigned transcripts remain in the data with a missing cell identifier.	Geometry is repaired only where required for valid spatial joins.

1.2.3 Node Feature Vectors

Segger's ability to make transcription-aware assignment decisions depends on equipping each node in the heterogeneous graph with a feature vector that encodes biologically meaningful information — not just identity or position, but something that reflects the transcriptional state of the entity the node represents. For cell nodes, the relevant information is where the cell sits on the phenotypic manifold: which cell state it represents, and how similar it is to other cells. For transcript nodes, the relevant information is what co-expression context the gene belongs to: which other genes tend to be expressed in the same cells, and how similar those co-expression patterns are across genes. These two feature spaces are constructed independently from the initial nuclear segmentation, without an external reference, and define the similarity structure that the cell and transcript embedding loss functions will subsequently enforce. The cell feature vectors additionally serve as the input features of cell nodes, whereas transcript nodes use a separately learned gene embedding (see below).

Cell feature vectors Cell node feature vectors are constructed from the initial nuclear segmentation. Transcripts overlapping each nuclear boundary are aggregated into a single-cell count matrix; genes with fewer than 100 nuclear transcripts are excluded to ensure stable similarity estimates; these genes are removed from the graph and do not appear in the output. Counts are library-size normalized to the median total count across cells passing a minimum count filter of 10 transcripts. The normalized count matrix is projected via PCA (GPU implementation in cuML with a fixed random seed, fitted on cells passing the count filter) to a low-dimensional representation equal to the input dimension of the GNN (default 128 dimensions), and all cells are projected into this learned space, including filtered cells. This gives each cell node an initial feature vector encoding its transcriptional state. Inside the encoder, this vector is linearly projected to d_{in} (Section 1.3.2); it is a fixed input feature rather than a learned initialization.

To define the similarity structure used during training, cells are clustered using Louvain community detection on a Jaccard-weighted k-nearest neighbor (kNN) graph (cuGraph v24.10, $k = 10$, resolution = 2.0). Rather than computing pairwise similarities between individual cells — intractable at the scale of 10^5+ cells per dataset — the similarity between two clusters is computed as the inner product of their mean L_2-normalized embeddings, $\mathbf{S}_{bd}[c, c'] = \bar{\mathbf{u}}_c^\top \bar{\mathbf{u}}_{c'}$ with $\bar{\mathbf{u}}_c = |c|^{-1} \sum_{k \in c} \mathbf{u}_k / \|\mathbf{u}_k\|_2$, where $\mathbf{u}_k$ is the PCA embedding of cell k and the mean is not renormalized. These similarities are stored in a cluster-level similarity matrix $\mathbf{S}_{bd} \in \mathbb{R}^{C \times C}$, where C is the number of clusters. This matrix defines the target similarity structure that the cell embedding loss $\mathcal{L}_{bd}$ enforces during training: cells of similar transcriptional state should occupy nearby positions in the learned latent space, and dissimilar cells should be pushed apart. Clusters with fewer than 100 cells are assigned to a background cluster

and excluded from $\mathcal{L}_{bd}$, as their mean embeddings are too sparse to yield stable similarity estimates. Library-size normalization, PCA, and clustering are computed entirely from the training data; no external reference atlas is used.

Optional geometric features As an alternative to the expression embedding, each cell node can be represented by four scalar descriptors of its boundary polygon B, computed with GeoPandas: area $A(B)$; convexity $A(\text{ConvexHull}(B))/A(B)$; elongation $A(\text{MBR}(B))/A(\text{envelope}(B))$, the ratio of the minimum-rotated-rectangle area to the axis-aligned bounding-box area; and circularity $A(B)/r_{\min}^2$, where $r_{\min}$ is the radius of the smallest enclosing circle. This geometry vector replaces the expression embedding only when explicitly requested. It is used in the loss ablation, so that a model trained without the cell embedding loss does not receive expression information about cells through its inputs (**Extended Data Fig. 1c**).

Transcript feature vectors Transcript node feature vectors encode gene co-expression structure. A gene–gene similarity matrix is computed from the library-size-normalized count matrix of the nuclear segmentation using Pearson correlation between gene expression profiles across cells. Because the embedding dimension equals the number of genes, reaching above 20,000 in whole-transcriptome platforms such as CosMx, this matrix is projected to d_{in} dimensions (default 128) via PCA using scikit-learn v1.8 with a fixed random seed, typically retaining 70–80% of variance, yielding a compact embedding vector for each gene. These gene embeddings define the gene-cluster similarity prior described below but are not used as transcript input features. Transcript nodes instead use a learnable gene-token embedding table of size $n_{\text{genes}} \times d_{in}$ that is randomly initialized and trained jointly with the network; all transcripts of the same gene start from the same token embedding, and message passing makes their representations context dependent (Section 1.3.3).

As with cell nodes, computing triplet losses directly at the level of individual transcripts is computationally intractable: iST datasets contain orders of magnitude more transcripts than cells, making per-transcript online triplet mining prohibitively slow, particularly for large gene panels. Genes are therefore grouped into clusters using Louvain community detection on a Jaccard-weighted kNN graph ($k = 5$, resolution $= 2.0$), and inner products between the cluster means of the L_2-normalized gene embeddings, computed as for cells, are stored in a cluster-level similarity matrix $\mathbf{S}_{tx} \in \mathbb{R}^{G \times G}$, where G is the number of gene clusters. Unlike cell-node clustering, no minimum cluster size filter is applied: small gene clusters likely represent genuine co-expression modules, and excluding them would leave some genes without triplet targets during training (see Section 1.3.3). The architectural strategy of reducing to cluster-level similarity matrices is shared between cell and transcript feature construction, but the two operate on entirely separate entities, separate latent spaces, and separate clustering solutions.

Optionally, the initial gene clustering and similarity matrix can be derived from an external scRNA-seq reference rather than from the nuclear segmentation itself, which may improve representation of rare cell types whose marker genes are underrepresented in the ST probe set or yield more faithful gene-gene similarities when the iST data suffer from high transcript misassignment (e.g., spillover of transcripts into neighboring cells), which distorts the co-expression structure estimated from the segmentation. This option requires careful consideration as scRNA-seq gene panels typically do not match the ST probe set, and is not used in the analyses presented in this manuscript.

1.2.4 Graph edges

The two edge types in Segger's heterogeneous graph serve complementary roles: transcript–transcript edges propagate local transcriptional context among neighboring transcripts, while transcript–cell edges define the candidate assignment structure that the model learns to refine.

Transcript–transcript edges Transcript–transcript edges $\mathcal{E}_{tx-tx}$ connect each transcript to its k_{tx} nearest spatial neighbors within a maximum distance d_{tx}, constructed via radius-restricted kNN search using a KD-tree (SciPy v1.16, leaf size 100). The two constraints serve distinct purposes: the neighbor limit k_{tx} prevents excessive connectivity in transcript-dense regions, effectively normalizing for local transcript density, while the distance cutoff d_{tx} prevents connections from spanning cell boundaries in transcript-sparse regions. Together they ensure that message passing remains local — consistent with the Markov Blanket assumption that a transcript's identity is determined by its immediate spatial neighborhood — without creating spurious long-range connections. The default query includes the transcript itself, giving at most four other transcript neighbours and one self-loop ($k_{tx} = 5$, $d_{tx} = 5$ μm); guidelines for tuning are discussed in Section 1.7.

Segmentation edges Segmentation edges $\mathcal{E}_{seg}$ connect each transcript to the nuclear boundary it spatially overlaps, formally defined as:

$$\mathcal{E}_{seg} = \{(t_i, c_j) \mid (p_{i,x}, p_{i,y}) \in \text{int}(B_j)\}, \tag{1}$$

where $(p_{i,x}, p_{i,y})$ denotes the 2D coordinates of transcript t_i and $\text{int}(B_j)$ denotes the interior of the nuclear boundary polygon of cell c_j. These edges define positive training examples for the segmentation loss $\mathcal{L}_{seg}$: transcript–cell pairs connected by $\mathcal{E}_{seg}$ are the assignments the model is trained to reproduce and refine. Despite the contamination inherent in nuclear overlap assignments, these edges contain sufficient signal for the model to learn from and subsequently correct.

In practice, many iST platforms pre-compute transcript–cell assignments and provide them directly as transcript labels, removing the need to recompute these assignments via costly geometric comparisons. In this manuscript, $\mathcal{E}_{seg}$ is derived from the transcript-to-cell assignments provided as transcript labels in Xenium datasets, restricted to transcripts overlapping nuclear segmentation boundaries (on other platforms, the analogous assignment and compartment fields are used; Supplementary Table 1). At inference, a distinct and expanded set of candidate edges is constructed to allow the model to assign transcripts beyond the nuclear boundary; this is described in Section 1.5.

Positive and negative training examples Positive transcript–cell training pairs are read from the standardized input assignment, that is, the per-transcript cell identifier and compartment label (Supplementary Table 1), rather than recomputed by a geometric search, so that the same rule applies on every platform. The training-graph mode determines which pairs are used. In the default `nucleus` mode, nuclear transcripts are paired with the node of their cell; in the alternative `cell` mode, nuclear and cytoplasmic transcripts are paired with the corresponding whole-cell node. Positive pairs serve both as the message-passing edges $\mathcal{E}_{seg}$ and as positive examples for the segmentation loss $\mathcal{L}_{seg}$ (Section 1.3.3).

For each positive pair (t_i, c_{j^+}), exactly one negative cell c_{j^-} is sampled, so that positive and negative pairs are balanced one to one. With the B cell nodes of the current mini-batch indexed $0, \ldots, B-1$,

$$j^- = \left(j^+ + r\right) \bmod B, \qquad r \sim \text{Unif}\{1, \ldots, B-1\}, \tag{2}$$

where $r \neq 0$ guarantees $j^- \neq j^+$. The negative is therefore drawn uniformly from all other cell nodes of the same mini-batch, which typically combines several spatial tiles (Section 1.4.2), rather than from the full tissue section.

1.3 Segger model

Segger's core task is to learn a shared latent space in which transcripts and cells are embedded according to their transcriptional identity, such that a transcript's correct cell assignment corresponds to proximity in that space. The heterogeneous graph constructed in Section 1.2 provides the substrate for this learning: transcript nodes carry gene identity and spatial position, cell nodes encode a low-dimensional representation of each cell's transcriptional state derived from its gene expression profile, and the edges between them define both the flow of information during message passing (transcript-transcript edges) and the assignment structure the model is trained to refine (segmentation edges).

Learning this space is made tractable by two structural properties of iST data that Segger exploits explicitly. First, the cell identity of a transcript is determined primarily by its immediate spatial context — the genes and cells around it — rather than by distant transcripts elsewhere on the slide. This locality assumption, analogous to a Markov Blanket — a transcript's identity is conditionally independent of the rest of the slide given its local neighborhood — justifies representing each transcript's context as a sparse local graph rather than attending to the full dataset, and it is the foundation on which both the graph construction and the model architecture are built. Second, transcripts of co-expressed genes and cells of similar transcriptional state form structured manifolds in gene expression space that can be learned from the nuclear-segmentation expression profiles, without any external reference. Segger's loss functions are designed to discover and enforce this structure for transcripts and cells jointly, creating a shared embedding space in which assignment of transcripts to cells by similarity becomes meaningful.

1.3.1 Segmentation as link prediction

Segger frames cell segmentation as a link prediction task on the heterogeneous graph: rather than classifying transcripts into fixed cell type categories, the model learns to predict whether a transcript–cell edge should exist — that is, whether a given transcript belongs to a given cell. This framing is natural for the problem: Cell identity is not a fixed label attached to a transcript, but a relational property that depends on which cell a transcript is most consistent with given its gene identity and spatial context. Link prediction captures this relational structure directly, and allows the model to make assignment decisions that are sensitive to the specific transcriptional state of each cell in the dataset rather than to a fixed set of pre-defined categories.

To score the association between transcript t_i and cell c_j, Segger uses cosine similarity between their learned embeddings:

$$s_{ij} = \frac{\mathbf{z}_i^{tx} \cdot \mathbf{z}_j^{bd}}{\|\mathbf{z}_i^{tx}\| \cdot \|\mathbf{z}_j^{bd}\|}, \tag{3}$$

where $\mathbf{z}_i^{tx} \in \mathbb{R}^{d_{out}}$ and $\mathbf{z}_j^{bd} \in \mathbb{R}^{d_{out}}$ are the learned embeddings for transcript i and cell j respectively. Output embeddings are $L2$-normalized to the unit hyper-sphere, making the dot product exactly equal to cosine similarity and bounding association scores to $[-1, 1]$ regardless of embedding magnitude. This normalization prevents representation collapse, ensures that scores reflect the direction of embeddings rather than their scale, and makes the similarity measure invariant to factors that affect embedding magnitude but not biological identity. Because the embeddings lie on the unit sphere, Euclidean distance is a monotone function of cosine similarity, $\|\mathbf{z}_a - \mathbf{z}_b\|_2^2 = 2\big(1 - \cos(\mathbf{z}_a, \mathbf{z}_b)\big)$, so that the triplet margin losses in Section 1.3.3, which are formulated on Euclidean distances, act directly on the cosine scores used for assignment.

Positive training examples are transcript–cell pairs (t_i, c_j) where transcript t_i spatially overlaps cell c_j in the nuclear segmentation, as defined by $\mathcal{E}_{seg}$ (Section 1.2.4). These pairs are imperfect positives: as established in Section 1.2, the nuclear segmentation contains contaminating assignments that do not reflect true cell membership. The loss functions in Section 1.3.3 are designed explicitly to learn from and correct these noisy positive examples rather than treating them as ground truth, and the choice of negative sampling strategy is motivated by the same consideration.

1.3.2 Segger architecture

Motivation for graph attention The Markov Blanket assumption established in Section 1.3 has a direct architectural implication: The model should attend selectively to local neighborhoods, not to the full dataset. Transformer self-attention, while state-of-the-art for learning conditional relationships between elements in a set, scales quadratically $\mathcal{O}(N^2)$ in sequence length, making it computationally infeasible for the millions of transcripts in a typical iST dataset. A Graph Attention Network (GAT) resolves this by enforcing locality directly into the architecture: Rather than computing attention over all possible pairs, it restricts attention to edges in the sparse local graph, which is precisely the neighborhood structure the Markov Blanket assumption requires. Transformer self-attention is in fact a special case of GAT attention for fully connected graphs [1]; Segger exploits the geometric structure of iST data to replace the full attention matrix with a sparse one, gaining both computational tractability and biological interpretability.

Segger uses GATv2 [2] rather than the original GAT [3]. Standard GAT computes attention coefficients from a static, separable combination of source and target node features, which limits its expressiveness in settings where the relationship between two nodes depends on their joint configuration. GATv2 introduces dynamic attention, in which coefficients are computed as a function of both source and target features after a shared linear transformation. This is important in Segger's heterogeneous setting, where transcript and cell nodes have fundamentally different feature types: the same transcript may need to attend differently to cells of different transcriptional states, a conditional relationship that static attention cannot capture.

Heterogeneous message passing Segger's graph contains two node types — transcripts and cells — with different feature spaces and different biological meanings. A homogeneous GNN would apply the same transformation to both, collapsing the distinction between gene identity and cell state. Segger instead uses a heterogeneous GATv2, implemented as a `HeteroConv` module (PyTorch Geometric v2.7 [4]) that applies independent `GATv2Conv` modules for each active edge type:

- $\mathcal{E}_{tx-tx}$: transcript-to-transcript message passing, propagating local transcriptional context among spatially proximate transcripts. The `GATv2Conv` operators are configured with `add_self_loops=False`; the transcript–transcript graph nonetheless includes the query transcript itself among its k nearest neighbours (Section 1.2). Thus $k = 5$ includes at most four other transcripts, and the convolution does not insert an additional self-loop.
- $\mathcal{E}_{seg}$: transcript-to-cell message passing, where transcripts propagate information to their nuclear-assigned cell node.

A third, reverse cell-to-transcript operator is defined in the encoder but receives no edges, so messages never pass from cell nodes back to transcripts; relationships among cells are imposed only through the cell embedding loss $\mathcal{L}_{bd}$ (Section 1.3.3). Messages from different edge types arriving at the same node are aggregated by summation. A dropout rate of 0.2 is applied within each `GATv2Conv` layer.

Positional encoding Spatial coordinates are encoded using a sinusoidal positional encoding [5] applied independently to each coordinate axis. For a scalar position x, the encoding produces a 256-dimensional frequency vector:

$$\cos(x \cdot f_0), \sin(x \cdot f_0), \ldots, \cos(x \cdot f_{127}), \sin(x \cdot f_{127}), \tag{4}$$

where $f_k = \exp(-\log(T) \cdot k/128)$ for $k = 0, \ldots, 127$, with maximum period $T = 10{,}000$, giving frequencies ranging from 1 to 1/10,000 in log-uniform steps. Each 256-dimensional encoding is passed through a two-layer MLP with SiLU activation that maps it to $d_{in}/2 = 64$ dimensions; outputs for the x and y axes are concatenated to yield

a 128-dimensional positional vector. SiLU is used in the positional MLP as a smooth activation well-suited to transforming continuous coordinate values.

Input coordinates are in micrometers on all platforms (CosMx pixel coordinates are converted during preprocessing) and are normalized per tile to $[0, 1]$ using the spatial extent of all nodes within that tile (see Section 1.4.1). Normalization happens after graph construction and before encoding, so that it does not affect edge connectivity. Positional embeddings are concatenated with the node feature embeddings (each of dimension d_{in}), yielding an effective input dimension of $2 \cdot d_{in} = 256$ to the first graph convolution layer.

Overall layer architecture Intuitively, through Segger's GNN, each transcript gathers information from nearby transcripts, and each cell node gathers information from the transcripts assigned to it. Rather than each transcript or cell making an isolated decision based solely on its own features, every transcript "listens" to its neighbors and every cell to its assigned transcripts. In particular, the attention mechanism within `GATv2Conv` layers assigns weights to these incoming messages, effectively highlighting the more informative signals — such as a strong gene identity match or transcriptional similarity with a cell — while down-weighting less relevant ones. However, this process must be carefully moderated, as excessive message passing can lead to over-smoothing, where distinctive features become diluted. Consequently, Segger typically uses a moderate number of layers to enhance transcript-to-cell assignment while preserving the unique identity of each node (see Section 1.7).

- *Input projection:* The gene embedding lookup table maps each transcript's gene index to a d_{in}-dimensional feature vector. A linear layer maps cell node PCA features to the same d_{in} dimensions. Positional embeddings (d_{in} dimensions) are concatenated with the projected node features, giving $2 \cdot d_{in} = 256$-dimensional inputs. GeLU activation is applied throughout the main network.
- *Message passing:* The Segger GNN includes $L + 2$ `GATv2` convolution layers in total — one input layer ($2 \cdot d_{in} \to d_{hid} \times H$), L middle layers ($d_{hid} \times H \to d_{hid} \times H$), and one output layer ($d_{hid} \times H \to d_{out} \times H$), where H is the number of attention heads. Multi-head outputs are concatenated (the default in `GATv2Conv`), yielding $d_{hid} \times H$ or $d_{out} \times H$ features after each layer.
- *Output projection:* A per-node-type linear layer maps $d_{out} \times H \to d_{out}$ for each node type independently, into a d_{out}-dimensional embedding space shared by transcripts and cells.
- *Normalization:* Output embeddings are L2-normalized to unit length, so dot products equal cosine similarities as defined in Section 1.3.1.
- *Default hyperparameters:* $d_{in} = 128$, $d_{hid} = 64$, $d_{out} = 64$, $L = 2$ middle layers, $H = 2$ heads. The encoder therefore comprises four GATv2 blocks. Guidelines for adjusting these parameters are discussed in Section 1.7.

1.3.3 Loss functions

Segger is trained using three coupled contrastive loss functions that together build a shared latent space in which transcript–cell assignment corresponds to proximity. Each loss performs a distinct and necessary role: $\mathcal{L}_{bd}$ discovers the structure of cell states — learning which cells are transcriptionally similar and which are distinct; $\mathcal{L}_{tx}$ discovers and preserves gene co-expression structure — learning which genes belong together and which are mutually exclusive, while allowing transcripts of the same gene to develop context-dependent representations through local message passing; and $\mathcal{L}_{seg}$ aligns these two independently learned manifolds into a single joint embedding space, pulling each transcript toward the cell it belongs to and away from cells it does not. The three losses are not interchangeable: ablating any one of them reduces the cell-type specificity of the learned transcript-to-cell similarities (Extended Data Fig. 1c), reflecting the fact that each is providing information the others do not.

Contrastive learning is the natural framework for this problem. Cell identity in tissue is not a discrete label but a position on a continuous phenotypic manifold — a manifold rich with transitional states, rare populations, and context-dependent variation that hard categorical labels collapse and destroy. Contrastive losses encode graded similarity relationships rather than forcing binary decisions, preserving the biological richness of the data. This is particularly important for the cross-manifold alignment performed by $\mathcal{L}_{seg}$: aligning a gene co-expression manifold to a cell state manifold does not require knowing exactly where each transcript should sit in absolute terms, only that it should be closer to its correct cell than to randomly sampled others — a relative objective that contrastive learning is ideally suited to.

The three losses are combined as a weighted sum:

$$\mathcal{L} = \tilde{w}_{tx}(t) \cdot \mathcal{L}_{tx} + \tilde{w}_{bd}(t) \cdot \mathcal{L}_{bd} + \tilde{w}_{seg}(t) \cdot \mathcal{L}_{seg}, \tag{5}$$

where $\tilde{w}_{tx}(t)$, $\tilde{w}_{bd}(t)$, and $\tilde{w}_{seg}(t)$ are epoch-dependent normalized weights that follow a curriculum schedule (see Section 1.3.3). The curriculum reflects the logical dependency between the losses: $\mathcal{L}_{bd}$ and $\mathcal{L}_{tx}$ run at full weight

from the start, building structured cell and gene manifolds, while $\mathcal{L}_{seg}$ is introduced gradually — alignment of two spaces is only meaningful once both spaces are organized.

Cell embedding loss $\mathcal{L}_{bd}$ The cell embedding loss is the loss that learns the biology. Its goal is not merely to separate cells of different types, but to reconstruct the full geometry of the cell state space in the learned latent representation — to produce embeddings in which the metric relationships between cells faithfully reflect their transcriptional similarity, as captured by gene expression. This is why Segger's learned latent space does not merely approximate the phenotypic manifold: it closely reflects it. In the non-diseased colon section used to follow the embeddings during training, the structure of Segger's cell embeddings matches the structure of the single-cell gene expression manifold (**Extended Data Fig. 1b**), a consequence of $\mathcal{L}_{bd}$ directly supervising the geometry of the embedding space rather than just its topology.

This geometric fidelity is why $\mathcal{L}_{bd}$ uses mean squared error (MSE) regression rather than a triplet margin loss. A triplet loss enforces only a rank ordering — that a positive example is closer to the anchor than a negative by some margin — but says nothing about the absolute distances between embeddings. MSE regression to the pre-computed cluster similarity matrix $\mathbf{S}_{bd}$ is strictly stronger: it requires the embedding space to be metrically faithful to the gene expression similarity structure, not just topologically consistent with it. The target similarities in $\mathbf{S}_{bd}$ are not labels imposed from outside the data — they are derived from the gene expression profiles of the cells themselves, computed directly from the nuclear segmentation. $\mathcal{L}_{bd}$ asks the learned space to reproduce this structure, making the embedding space a reflection of the biology encoded in the data.

A second reason for explicit regression targets is specific to Segger's graph structure. In the heterogeneous graph, cells have no edges to other cells: a cell node's only incoming messages during message passing come from its assigned transcripts via $\mathcal{E}_{seg}$. This means that without $\mathcal{L}_{bd}$, cell embeddings have no mechanism to learn how cells relate to each other — their representations would be determined entirely by the transcript signals they happen to receive, with no guarantee of biological coherence across the dataset. $\mathcal{L}_{bd}$ is the only place in the model where cell–cell relationships are explicitly supervised, making it the architectural foundation on which the rest of the loss framework depends.

Triplets for $\mathcal{L}_{bd}$ are sampled using a cluster-weighted mechanism that we describe here, and the same mechanism is used for $\mathcal{L}_{tx}$ (see below). Cells are grouped into clusters via Louvain community detection on the PCA representation of the nuclear count matrix (Section 1.2.3), and cluster similarities (Section 1.2.3) are stored in $\mathbf{S}_{bd} \in \mathbb{R}^{C \times C}$. The sampler sets the diagonal of $\mathbf{S}_{bd}$ to one (unit self-similarity), so that the anchor's own cluster is a likely source of positives. For each anchor cell a with cluster label c_a, a positive cluster c^+ is sampled with probability proportional to $\mathbf{S}_{bd}[c_a, c^+]$ — clusters more similar to the anchor are more likely to provide positives — and a positive cell p is sampled uniformly from c^+. A negative cluster c^- is sampled with probability proportional to $-\mathbf{S}_{bd}[c_a, c^-]$, clamped to a minimum of $\varepsilon = 10^{-8}$ to exclude clusters with positive similarity from negative sampling, and a negative cell n is sampled uniformly from c^-. This cluster-weighted scheme focuses the gradient on informative triplets — avoiding trivially easy pairs where positives and negatives are already well-separated — while remaining tractable at the scale of 10^5+ cells where per-cell online triplet mining would be prohibitively slow.

The loss supervises the full similarity structure simultaneously, computing MSE over both positive and negative pairs for each anchor:

$$\mathcal{L}_{bd} = \frac{1}{|\mathcal{V}_{bd}|} \sum_{a} \left[\Big(\cos(\mathbf{z}_a, \mathbf{z}_p) - \max\big(\mathbf{S}_{bd}[c_a, c_p], \varepsilon\big)\Big)^2 + \Big(\cos(\mathbf{z}_a, \mathbf{z}_n) - \max\big(\mathbf{S}_{bd}[c_a, c_n], \varepsilon\big)\Big)^2 \right], \tag{6}$$

where $\mathbf{z}_a$, $\mathbf{z}_p$, $\mathbf{z}_n$ are the L2-normalized embeddings of the anchor, positive, and negative cell nodes respectively. Prior similarities are truncated below at $\varepsilon = 10^{-8}$. Unlike a standard triplet loss that only constrains the relative ordering of distances, this formulation simultaneously pulls similar cells to the correct distance from each other and pushes dissimilar cells towards orthogonality: because negatives are drawn from clusters with negative prior similarity and prior similarities are truncated at ε, the effective regression target for negative pairs is zero. Only cell nodes passing the minimum transcript count filter (≥ 10 transcripts) and having a valid cluster assignment contribute to this loss; cells below this threshold have insufficient transcript coverage for stable embedding estimation.

Transcript embedding loss $\mathcal{L}_{tx}$ The transcript embedding loss learns the gene co-expression structure of the data — which genes tend to be expressed together and which are mutually exclusive — and encodes this structure into the transcript embedding space. Unlike $\mathcal{L}_{bd}$, which uses MSE regression to reconstruct a fixed manifold geometry, $\mathcal{L}_{tx}$ uses a triplet margin loss that enforces relative similarity relationships while allowing geometric flexibility. This flexibility is essential for two reasons. First, gene pleiotropy means that many genes are expressed across multiple cell types and therefore cannot be assigned a single fixed position in embedding space — their correct representation depends on context. Second, and more fundamentally, the gene co-expression manifold and the cell state manifold do not have globally identical geometry. Forcing metric faithfulness on both

simultaneously would require the two spaces to align rigidly before $\mathcal{L}_{seg}$ has had the opportunity to learn their alignment — an overconstrained problem that would prevent the model from converging. The triplet margin loss preserves the relative structure of gene co-expression while leaving the geometric degrees of freedom that $\mathcal{L}_{seg}$ needs to align the two manifolds.

This flexibility also enables something biologically important: context-specific gene–gene covariance. Gene co-expression relationships are not globally fixed — the covariance structure among genes in an epithelial context differs from that in an immune or stromal context, and these context-specific relationships carry biological meaning that a globally rigid embedding would suppress. Through local message passing, each transcript attends to the genes in its immediate spatial neighborhood, allowing the model to learn and adapt to the covariance structure of different biological contexts rather than collapsing everything onto a single global prior. $\mathcal{L}_{tx}$ anchors this learning to the global co-expression structure encoded in $\mathbf{S}_{tx}$, preventing local message passing from drifting arbitrarily, while permitting the contextual variation that makes transcript representations genuinely adaptive to the tissue environment they inhabit.

Gene clusters are constructed using the same Louvain community detection procedure applied to cells (Section 1.2.3), with a $k = 5$ nearest-neighbour graph, but with one important difference: no minimum cluster size filter is applied. Small gene clusters are likely to represent genuine co-expression modules — rare but biologically meaningful gene programs — and excluding them would leave some genes without triplet targets during training, breaking the supervision signal for those genes entirely. Cluster similarities, computed as for cells (Section 1.2.3), are stored in $\mathbf{S}_{tx} \in \mathbb{R}^{G \times G}$, where G is the number of gene clusters.

Triplets are sampled from $\mathbf{S}_{tx}$ using the same cluster-weighted mechanism described for $\mathcal{L}_{bd}$: for each anchor transcript a with cluster label c_a, a positive transcript p is sampled from a cluster drawn with probability proportional to $\mathbf{S}_{tx}[c_a, c^+]$, and a negative transcript n is sampled from a cluster drawn with probability proportional to $-\mathbf{S}_{tx}[c_a, c^-]$, clamped to a minimum of $\varepsilon = 10^{-8}$ to exclude clusters with positive similarity from negative sampling. Sampling is performed in-batch during training; the scalability challenges of triplet mining at the scale of millions of transcripts across spatially tiled mini-batches are discussed in Section 1.4.3. The loss is the standard triplet margin loss:

$$\mathcal{L}_{tx} = \frac{1}{|\mathcal{V}_{tx}|} \sum_{a} \max\left(0,\ \|\mathbf{z}_a - \mathbf{z}_p\| - \|\mathbf{z}_a - \mathbf{z}_n\| + m_{tx}\right), \tag{7}$$

where $m_{tx} = 0.3$ is the margin. The mean is taken over transcripts with a valid gene-cluster label. The value $m_{tx} = 0.3$ is the released default; models retrained with other margins are compared in **Extended Data Fig. 1d–f**.

Segmentation loss $\mathcal{L}_{seg}$ The segmentation loss is architecturally distinct from $\mathcal{L}_{bd}$ and $\mathcal{L}_{tx}$ in two fundamental ways. First, it is the only loss that operates across node types: where the previous two losses each organized one manifold independently, $\mathcal{L}_{seg}$ is the bridge that pulls the cell state manifold and the gene co-expression manifold into a single joint embedding space, making transcript-to-cell assignment by cosine similarity possible. Second, it is the only loss that directly incorporates the spatial structure of the data. The positive examples are edges in $\mathcal{E}_{seg}$ — transcript–cell pairs defined by physical co-localization in tissue space — making $\mathcal{L}_{seg}$ the point at which the spatial organization of the tissue enters the model as a direct training signal, not merely as a graph construction choice.

This spatial grounding is also what completes the context-specific transcript representations described above. $\mathcal{L}_{tx}$ creates the representational flexibility that allows transcripts of the same gene to develop different embeddings in different biological contexts, but it is $\mathcal{L}_{seg}$ that fills that flexibility with spatial meaning. A transcript's embedding is shaped by which cell it is assigned to in physical space, and the covariance structure it learns from its local neighborhood is anchored to a real spatial context in the tissue. The two triplet losses work in concert: $\mathcal{L}_{tx}$ creates the capacity for contextual variation, $\mathcal{L}_{seg}$ grounds that variation in the spatial reality of the data.

The positive training signal is real but imperfect. Positive edges are derived from nuclear overlap assignments, which are structured and biologically meaningful but can contain contaminating assignments that do not reflect true cell membership. The majority of positive edges carry genuine transcript–cell relationships, providing sufficient signal for the model to learn correct assignments despite the noise — a robustness that is further supported by the curriculum scheduling that limits $\mathcal{L}_{seg}$'s weight during training (see Section 1.3.3).

Negative examples are sampled uniformly at random from the other cell nodes of the same mini-batch (Eq. 2), rather than as hard negatives drawn from spatially proximate cells. This is a deliberate design choice: nearby cells in tissue tend to be of similar transcriptional type, so hard negatives sampled from the local neighborhood would introduce systematic bias — the model would be penalized for correctly recognizing transcriptional similarity between neighboring cells. Uniform random sampling gives the loss a clean and unbiased objective: a transcript's embedding should be closer to its assigned cell than to a randomly chosen cell of the mini-batch. Because a mini-batch combines tiles from several tissue regions (Section 1.4.2), the sampled negatives span multiple regions, but they are not drawn from the whole section, and a fraction of them lie in the same tile as the anchor transcript.

The loss is a triplet margin loss over transcript–cell pairs:

$$\mathcal{L}_{seg} = \frac{1}{N_{seg}} \sum \max\left(0,\, \|\mathbf{z}_i^{tx} - \mathbf{z}_{j^+}^{bd}\| - \|\mathbf{z}_i^{tx} - \mathbf{z}_{j^-}^{bd}\| + m_{seg}\right), \tag{8}$$

where $\mathbf{z}_i^{tx}$ and $\mathbf{z}_j^{bd}$ are the L2-normalized embeddings of transcript i and cell j respectively, N_{seg} is the number of positive segmentation edges, and $m_{seg} = 0.4$ is the segmentation margin. The loss is set to zero for a mini-batch containing at most one cell node. A binary cross-entropy objective on the transcript–cell dot-product logit is implemented as an alternative but is not used by default. The margin is set larger than the transcript embedding margin ($m_{seg} = 0.4$ versus $m_{tx} = 0.3$) because the segmentation loss faces a harder discrimination objective: it must separate each transcript from randomly sampled cells of the mini-batch, rather than from cluster-structured negatives where positives and negatives already differ substantially in embedding space. A wider margin maintains a proportionally stronger gradient through the segmentation objective. The value $m_{seg} = 0.4$ is the released default; a sweep around this value is shown in **Extended Data Fig. 1d–f**.

Loss weight scheduling The epoch-dependent loss weights are not merely a technical implementation choice — they reflect the logical dependency structure of the three losses themselves. The cell and gene manifolds must exist before they can be aligned: $\mathcal{L}_{bd}$ and $\mathcal{L}_{tx}$ therefore run at full weight from the first epoch, building the cell state and gene co-expression manifolds respectively. $\mathcal{L}_{seg}$ starts at zero weight and is introduced gradually as training progresses — alignment only becomes a meaningful objective once both spaces are sufficiently organized. Introducing $\mathcal{L}_{seg}$ too early would ask the model to bridge two spaces that do not yet have coherent structure, producing unstable gradients and biologically meaningless alignment. The curriculum is therefore not an optimization heuristic but a reflection of the causal order in which the biology must be learned.

The maximum normalized weight of $\mathcal{L}_{seg}$ at convergence is capped at $0.5/(1 + 1 + 0.5) \approx 0.20$. This cap serves two purposes. First, it prevents overfitting to the noisy positive edges in $\mathcal{E}_{seg}$: a segmentation loss that is too strong would force the model to reproduce contaminated assignments rather than learn to correct them. Second, and more fundamentally, it prevents $\mathcal{L}_{seg}$ from dominating and collapsing the manifold structure that $\mathcal{L}_{bd}$ and $\mathcal{L}_{tx}$ built. If alignment pressure were too strong, it would force the gene and cell manifolds to conform to each other so rigidly that the rich within-manifold structure — the cell state relationships and gene co-expression modules that carry the biological signal — would be destroyed. The cap ensures that alignment happens in service of the biology, not at its expense.

The weights follow a cosine annealing schedule, chosen as a standard smooth interpolation between start and end values. At epoch t:

$$\alpha(t) = \frac{1}{2}\left(1 + \cos\left(\pi \cdot \frac{\min(t, T-1)}{T-1}\right)\right), \tag{9}$$

$$w_k(t) = w_k^{end} + \left(w_k^{start} - w_k^{end}\right) \cdot \alpha(t), \tag{10}$$

with weights normalized to sum to 1:

$$\tilde{w}_k(t) = w_k(t) \Big/ \sum_k w_k(t). \tag{11}$$

Default values are $w_{tx}^{start} = w_{tx}^{end} = 1.0$, $w_{bd}^{start} = w_{bd}^{end} = 1.0$, $w_{seg}^{start} = 0.0$, $w_{seg}^{end} = 0.5$, implementing the curriculum described above: cell and gene embedding losses run at constant full weight throughout training, while the segmentation loss is introduced gradually from zero and plateaus at a normalized weight of approximately 0.20 at convergence.

Training converges within $T = 20$ epochs — remarkably few for a model learning to jointly embed two biological manifolds and align them. Two features of the setup contribute to this rapid convergence. First, cell nodes receive their PCA expression embedding (Section 1.2.3) as input features, so that transcriptionally similar cells start close to each other in the embedding space. Second, the similarity targets $\mathbf{S}_{bd}$ and $\mathbf{S}_{tx}$ are computed from the training data before training begins, so that the cell and transcript embedding losses fit a fixed, pre-computed structure rather than having to discover it. The gene-token embedding of transcript nodes is randomly initialized and acquires its co-expression structure through $\mathcal{L}_{tx}$ and message passing. The curriculum then orchestrates how training proceeds: first deepening the within-manifold structure of both the cell and gene spaces, then learning the cross-manifold alignment that makes transcript-to-cell assignment possible.

1.4 Training

Scaling Segger to iST datasets requires confronting a graph of unusual size. A single tissue section imaged with a modern iST platform may yield tens of millions to hundreds of millions of individually resolved transcript molecules, each represented as a node in the heterogeneous graph. Cell nodes, one per initial cell, add further entities. The radius-restricted kNN graph contributes up to k transcript–transcript edges per transcript, including its query self-loop, and the transcript–cell edges add further connections. Consequently, graph memory

requirements increase with the number of detected molecules and their local degree. **Extended Data Fig. 2a** places the node counts of 108 public iST sections alongside MAG240M and WikiKG90Mv2 from the Open Graph Benchmark Large-Scale Challenge [6]. Some whole-slide graphs approach or exceed the sizes of those established graph-learning benchmarks; the figure does not establish that most iST samples do so. Training a GNN on graphs of this scale cannot proceed by loading the full graph into GPU memory: a single sample's edge tensor alone would exceed the memory of most standard accelerators. The standard solution in the graph learning literature is mini-batching—sampling tractable subgraphs at each training step—but general-purpose graph samplers were designed for abstract connectivity structures and do not exploit the defining property of iST data: graph connectivity is determined entirely by spatial proximity, and therefore edge density is proportional to node density across the tissue. This geometric prior transforms the mini-batching problem from a graph-algorithmic one, which scales with edge count E, to a spatial one, which scales with node count N—a critical efficiency gain since $N \ll E$. The training strategy described in the following sections exploits this structure through a two-level approach: adaptive spatial tiling that partitions the tissue into compact, spatially coherent subgraphs (Section 1.4.1), followed by bin-packing-based batch assembly that fills GPU memory efficiently (Section 1.4.2). A third component addresses the particular challenge that local spatial tiling poses for contrastive learning (Section 1.4.3). Together, these components constitute a reusable toolkit for training GNNs on any large-scale spatial graph, with applicability beyond this iST segmentation setting.

1.4.1 Adaptive spatial tiling

Efficient training on iST-scale graphs requires partitioning the full heterogeneous graph into subgraphs that fit within GPU memory at each training step. The core challenge is not merely size but *boundary cost*: in a multi-layer GNN, computing the embedding of any node requires its k-hop neighborhood, so nodes near a partition boundary have incomplete neighborhoods and cannot contribute reliable gradients. Effective partitioning therefore demands maximizing intra-partition connectivity—equivalently, minimizing the fraction of edges that are cut at partition boundaries. Standard graph sampling strategies fail to achieve this on iST graphs. Neighborhood sampling (GraphSAGE [7]) fetches k-hop computation graphs per node, incurring a high fraction of boundary nodes by construction. Random subgraph sampling (GraphSAINT [8]) eliminates most of each node's spatial context. Community-based partitioning (Cluster-GCN [9]) uses graph-structure algorithms that scale with edge count E and yield little benefit when the graph lacks strong community structure—as is the case for the near-homogeneous local connectivity of transcript kNN graphs.

The key insight is that iST graphs possess a geometric prior absent from the generic graphs these methods were designed for. Transcript–transcript edges are constructed by radius-restricted kNN: each transcript connects to at most k spatial neighbors within distance r. When the radius constraint is not limiting — as is typical across most of the tissue — node degree is determined by k rather than local transcript density. This makes edge density proportional to node density across the tissue, a property that holds for random geometric graphs generally. The consequence is fundamental: the graph partitioning problem, which naively scales with edge count E, can be recast as a spatial partitioning problem scaling with node count N—and since $N \ll E$, this is a substantial gain in tractability.

This spatial reframing connects mini-batch efficiency directly to a classical result in geometry. Cluster-GCN established that minimizing inter-batch edge cuts is equivalent to maximizing embedding utilization during SGD. For random geometric graphs, graph isoperimetry shows that in expectation, internal edges scale with partition area and cut edges scale with partition perimeter. Minimizing the edge cut for fixed partition area is therefore equivalent to minimizing the perimeter-to-area ratio of spatial partitions—a well-studied geometric optimization. For uniform node density, the honeycomb theorem gives the exact optimum: a regular hexagonal tiling. For the non-uniform densities characteristic of real tissue (dense epithelial sheets, sparse stromal regions, void spaces between glands), the exact optimum corresponds to hexagonal tiling on a conformally mapped space of uniform density, which is NP-hard to compute. We therefore use a practical heuristic that approximates this optimum: an adaptive quadtree.

The quadtree recursively subdivides the spatial extent of all nodes—transcripts and cell centroids jointly—into quadrants, splitting until no leaf tile contains more than a specified maximum node count (default 50,000 nodes). This produces small, compact tiles in dense tissue regions and larger tiles in sparse regions, automatically adapting to tissue architecture without requiring knowledge of tissue structure in advance (**Extended Data Fig. 2b,c**). Because quadtree leaves are square and non-overlapping, partitions are geometrically compact by construction, approximating low perimeter-to-area ratios without explicit optimization. The quadtree is constructed using cuSpatial/cuGraph v24.10 with Morton code ordering and radix sort, achieving near-linear $O(N \log N)$ construction time. Node membership in a tile is determined by a GPU point-in-polygon join between node coordinates and the quadtree leaf polygons. Nodes not matched by the join are excluded from tiling, and if the decomposition does not index every point, tiling is repeated with leaf capacities between 1,000 and 500,000 nodes. The 50,000-node default is calibrated for GPUs with ≥ 40 GB memory; it should be reduced proportionally for smaller devices.

Within each tile, a strict spatial partition is enforced: nodes are assigned to the tile containing their coordinates, and edges crossing tile boundaries are removed. However, nodes near a tile boundary have truncated spatial neighborhoods and therefore incomplete GNN receptive fields. To address this, each tile is augmented with a margin zone: nodes within distance $\delta = 20$ μm of the tile boundary are included in message passing but excluded from the cell and transcript embedding losses, which are evaluated only on nodes in the interior region—the tile minus its margin. The segmentation loss uses all positive edges in the tile, with margin nodes serving as message-passing context. This decouples the receptive field problem from the partition boundary: interior nodes receive complete neighborhood context up to the margin width, while boundary artifacts do not propagate into the learned embeddings. During training, the interior region is obtained by negatively buffering each leaf polygon by the margin (square cap, mitre join); for small tiles that the full buffer would eliminate, the margin is halved progressively. During inference, tiles are instead buffered outward and only transcripts inside the original tile are emitted (Section 1.5.1). The margin of 20 μm is conservative relative to the 3-hop spatial extent of the graph. Estimated as $\max(2d + r, 3d)$ from the 75th percentiles d and r of transcript–transcript and transcript–cell edge lengths, this extent ranged from 4.3 to 6.5 μm across four Xenium datasets; with a margin of this size and the default tile capacity, 98.7–99.4% of nodes retained a complete receptive field and 78.4–86.5% remained available for training. Specifically, the 20-µm margin is chosen to be approximately equal to the diameter of a typical mammalian cell, ensuring that cell nodes and transcript nodes near tile boundaries have sufficient spatial neighborhood context during message passing while preventing boundary artifacts from influencing learned embeddings.

Transcript neighbourhoods are constructed once at data preparation with $k = 5$ and a maximum radius of 5 μm by default. The resulting edge set is held fixed during training; randomness between epochs comes from training-tile batch composition and negative and triplet sampling, not from rebuilding the transcript graph. This preserves the local spatial graph across epochs while avoiding repeated whole-section neighbour searches.

1.4.2 Mini-batch assembly

Spatial tiling (Section 1.4.1) partitions the tissue into spatially coherent subgraphs, but tiles are not themselves training batches. A single tissue section is rarely biologically uniform: epithelial glands, stromal compartments, immune infiltrates, and vascular structures each occupy distinct spatial territories with characteristic cell type compositions and gene expression programs. For the model to learn representations that generalize across the full biological diversity of a sample, each training batch must draw from multiple tissue regions simultaneously—a batch composed entirely of transcripts from a single dense epithelial region, for instance, would over-represent epithelial gene programs and provide no signal about stromal or immune cell identities during that step. At the same time, because transcript density varies greatly across tissue regions, tiles vary considerably in edge count even when their node counts are bounded by the quadtree capacity: dense epithelial regions produce substantially more edges per unit area than sparse stroma or acellular voids. Grouping a fixed number of tiles per batch would therefore produce highly variable GPU loads—some batches overflowing memory, others leaving the GPU severely underutilized—as well as biologically unbalanced training signal (**Extended Data Fig. 2d**, which shows the transcript-per-tile distributions under fixed versus adaptive tiling across platforms; with fixed tiling, 34–60% of occupied tiles exceed the configured capacity depending on the dataset—an upper bound on the fraction of tiles processed above the memory budget—whereas adaptive tiling bounds every tile by construction). Batch assembly is the second level of the training hierarchy, bridging spatially coherent tiles and computationally efficient GPU batches that capture the tissue's biological heterogeneity.

The binding constraint on batch size is edge count, not node count. During the forward and backward passes of a GATv2 layer, intermediate attention tensors are materialized for every edge in the batch; memory consumption therefore scales with the total number of edges present, not merely the number of nodes. We impose a maximum per-batch edge budget of 1,000,000 edges by default, which bounds the size of these tensors within the memory of a standard research GPU ($\geq$ 40 GB); this budget is a configurable parameter and should be reduced proportionally for smaller devices. The batch assembly problem is then precisely stated: given a collection of tiles with known edge counts, pack them into batches such that the total edge count of each batch is as close to the budget as possible without exceeding it. This is an instance of the classical bin-packing problem—tiles are items, their edge counts are item sizes, GPU batches are bins, and the bin capacity is the edge budget—which is NP-hard in general and therefore addressed with well-studied approximation algorithms.

For training, tile indices are shuffled and repacked at each epoch using first-fit packing under the edge budget. In Segger v0.2.0, the sampler calls `first_fit_decreasing_bucketed` with its default of one bucket: this shuffles the entire tile list before first-fit placement, rather than preserving a descending size order. Batch composition therefore varies across epochs, whereas the transcript-neighbour graph remains fixed. Each packed batch respects the configured edge budget; its tissue composition depends on the shuffled order of spatial tiles.

For validation, we use Best Fit Decreasing (BFD) bin packing, which places each tile into whichever existing batch has the least remaining capacity that can still accommodate it, producing deterministic and tightly packed batches. **Extended Data Fig. 2e** shows batch efficiency—the ratio of edges used to batch capacity—across four

strategies; BFD and FFD both approach near-perfect efficiency (median ≈ 1.0), while fixed-size batching and PyG's dynamic sampler leave substantial capacity unused. Determinism in validation is deliberate: monitoring the three loss components across epochs requires that validation batches remain identical from epoch to epoch, so that changes in loss reflect model learning rather than variation in batch composition. The train/validation split is a random partition of tiles (default 75% training, 25% validation), performed once before training begins. By default, we do not perform early stopping or checkpoint selection based on validation loss; the validation set serves as a diagnostic rather than a selection criterion. For inference, tiles are packed by greedy sequential accumulation, the strategy of the dynamic batch sampler in PyTorch Geometric [4]: tiles are streamed in a fixed order and a batch is closed when the next tile would exceed the edge budget.

Partition-based batch construction. Because the tile partition does not change during training, Segger materializes it once by permutation rather than repeatedly extracting subgraphs. The full section is held as a single PyTorch Geometric `HeteroData` object. Nodes of each type are sorted by tile label and edges by the tile of their source node; edges whose endpoints lie in different tiles are removed once, and compressed sparse row (CSR) pointers are stored for every node and edge type. A tile subgraph, with locally rebased indices, is then recovered as a contiguous slice of the sorted arrays, and a mini-batch is formed by concatenating the slices of the tiles selected by the bin-packing sampler. Batch construction therefore avoids evaluating a Boolean mask over the full graph and relabelling edge indices for every tile. Transcript and cell embeddings and the gene- and cell-cluster similarity priors are extracted once during partitioning. Compared with per-tile extraction using the PyTorch Geometric subgraph method, partition-based loading was 3 to 30 times faster, with larger gains on larger datasets, and used about 18-fold less peak GPU memory.

Model parameters are optimized using the Adam optimizer [10] with a fixed learning rate of 10^{-3} and $(\beta_1, \beta_2) = (0.9, 0.999)$. No learning rate scheduling is applied, because the loss weight curriculum described in Section 1.3.3 already provides structured emphasis shifts across training epochs—progressively increasing the weight of the segmentation loss as transcript and cell embeddings stabilize—making additional learning rate annealing redundant. Training runs for $T = 20$ epochs. No fixed random seed is set by default for tile shuffling, train/validation splits, or negative sampling and triplet mining; results may therefore differ across runs due to these stochastic components. By contrast, the principal-component analyses that define cell features and similarity priors and the subsampling used for per-gene threshold estimation use fixed seeds.

1.4.3 Training sample selection

Contrastive learning requires informative triplets: an anchor, a positive example that should be close to it in embedding space, and a negative example that should be far. What makes a triplet informative is the quality of the negative — a negative that is already far from the anchor in embedding space contributes no gradient and teaches the model nothing. The spatial tiling strategy described in Section 1.4.1 creates a structural tension with this requirement that must be resolved explicitly.

The tension arises from a property of tissue biology that the Markov Blanket assumption encodes: transcripts are locally homogeneous. Cells of the same type cluster together in tissue — epithelial cells line glands, immune cells infiltrate stroma, fibroblasts occupy connective tissue — and the transcripts they produce reflect this spatial organization. The same locality that justifies representing each transcript's context as a sparse local graph also means that transcripts within the same spatial tile tend to share similar gene expression programs. Spatially compact tiles, by construction, are transcriptionally homogeneous batches. This is precisely the condition under which contrastive learning degrades: if most transcripts in a batch belong to the same cell types, genuinely dissimilar negatives are rare or absent, the loss saturates on trivial triplets, and the gradient signal collapses. The Markov Blanket assumption that makes tiling biologically justified is the same property that makes naive in-batch negative sampling biologically uninformative.

The obvious remedy — pre-computing triplets across the full dataset before training — is infeasible at iST scale. The number of possible transcript triplets across 10^8 transcripts and thousands of genes is combinatorially intractable, and more fundamentally, it is incompatible with the stochastic batching strategy of Section 1.4.2: because tile groupings are randomized at every epoch, the set of transcripts present in any batch changes continuously, making pre-defined triplets invalid as soon as the batch composition changes.

The solution is in-batch probabilistic online triplet mining anchored to a global similarity prior. Rather than sampling negatives from the transcripts that happen to be present in the current batch — which are spatially proximate and therefore likely transcriptionally similar — triplets are defined at the level of genes, using the pre-computed cluster similarity matrix $\mathbf{S}_{tx}$ as the sampling distribution (Section 1.3.3). For each anchor transcript in the batch, a positive gene is sampled with probability proportional to its cosine similarity to the anchor gene, and a negative gene is sampled with probability proportional to its negative cosine similarity — effectively, the most dissimilar genes available. Crucially, $\mathbf{S}_{tx}$ is computed from the full dataset before training begins and encodes global gene–gene co-expression relationships across all cell types in the sample. It does not depend on which tiles happen to be in the current batch. This decouples the quality of the contrastive signal from the

local transcriptional composition of the batch: even in a spatially homogeneous batch dominated by epithelial transcripts, the similarity matrix supplies knowledge of which genes are characteristic of immune or stromal cells and samples them as negatives with high probability. The global prior compensates for the local impoverishment of the negative pool.

Scaling this procedure to large gene panels introduces a further challenge, making per-gene similarity computation and sampling prohibitively slow. We address this by reducing the gene–gene similarity matrix to a cluster–cluster similarity matrix: genes are grouped into a small number of clusters via Louvain community detection on their embedding similarities, pairwise cosine similarities are computed between cluster mean embeddings, and triplet sampling is performed at the cluster level rather than the gene level. A positive cluster is drawn with probability proportional to its similarity to the anchor's cluster, a positive gene is then sampled uniformly from that cluster, and the analogous procedure applies for negative sampling. This reduces the sampling cost from $\mathcal{O}(G^2)$ to $\mathcal{O}(G_{cl}^2)$, where $G_{cl} \ll G$ is the number of gene clusters, without discarding the global structure encoded in the similarity relationships. The same cluster-level architecture applies to the cell embedding loss $\mathcal{L}_{bd}$, where the number of cells per dataset — reaching 10^6 in large tissue sections — makes per-cell online triplet mining equally intractable; cluster-weighted sampling over the cell cluster similarity matrix $\mathbf{S}_{bd}$ provides the same solution in the cell manifold as it does in the gene manifold (Section 1.3.3).

This triplet mining strategy is not specific to iST segmentation. It is a general solution to a general problem: contrastive learning on any spatially tiled graph where local batches are locally homogeneous and therefore informationally impoverished for negative sampling. The combination of global similarity priors, cluster-level reduction, and online in-batch sampling provides informative gradients at every training step regardless of batch composition, and scales gracefully from small targeted gene panels to whole-transcriptome spatial data.

1.5 Inference and post-processing

Once training is complete, Segger has learned a shared latent space in which transcripts and cells are jointly embedded according to transcriptional identity: co-expressed genes occupy nearby positions, transcriptionally similar cells cluster together, and — through the alignment enforced by $\mathcal{L}_{seg}$ — a transcript's embedding reflects which cell it most likely belongs to. Inference is the process of applying this learned space to produce a complete segmentation of the tissue.

The inference problem differs from the training problem in a fundamental way. During training, the model saw only transcripts with high-confidence nuclear assignments — the positive edges of $\mathcal{E}_{seg}$ — and learned from these what correct transcript–cell associations look like in embedding space. At inference, the model must assign every transcript in the tissue, including the cytoplasmic transcripts that were deliberately excluded from training supervision. Segger recovers them not by expanding boundaries geometrically, but by asking whether each transcript's embedding is consistent with a nearby cell's transcriptional identity — the same question the model was trained to answer, now applied beyond the nuclear boundary.

Two design principles govern inference. First, candidate cells for each transcript are restricted to a spatial receptive field around the transcript's physical location, ensuring that assignment decisions respect the spatial organization of the tissue and preventing transcriptionally similar but physically distant cells from capturing transcripts across the slide. Second, every assignment is accompanied by a cosine similarity score that quantifies how confidently the transcript's embedding matches its assigned cell. Alongside the retained assignments, Segger reports every assignment with its score, enabling downstream analyses to apply whatever confidence criteria are appropriate for their biological question. By default, a per-gene adaptive threshold determines which assignments are retained in the Segger segmentation and its cell-by-gene matrix (Eq. 13); because all scores are stored, the segmentation can be re-thresholded without retraining. The following sections describe each step in detail.

1.5.1 Assigning transcripts to cells

At inference, transcript-to-cell assignment is determined by the conjunction of two complementary constraints: the learned similarity in the joint embedding space encodes what a correct assignment looks like transcriptionally, and the hard geometric receptive field enforces where a correct assignment is physically possible. Neither constraint alone is sufficient. Cosine similarity without spatial constraint would allow a transcript to be assigned to a transcriptionally similar cell anywhere in the tissue. Spatial constraint without learned similarity would reproduce the failures of purely geometric segmentation methods — assigning transcripts based on boundary proximity rather than transcriptional consistency. It is the conjunction of both that makes Segger's assignments spatially coherent and transcriptionally meaningful. Crucially, because assignment is determined by transcriptional similarity rather than geometric containment, transcripts that are physically inside a nuclear boundary but transcriptionally inconsistent with that cell do not automatically win their assignment — they must compete on the same terms as every other transcript, and may be outcompeted by a neighboring cell or left unassigned. Contamination removal is not a post-processing step: it is an inherent consequence of framing segmentation as an assignment problem rather than a boundary problem. Expansion-based segmentation makes two decisions with one geometric rule: it sets the extent of each cell by expanding its mask, and it assigns every transcript

inside the expanded mask to that cell. Segger separates these decisions—geometry determines only which cells are plausible candidates for a transcript, and molecular context determines which candidate it belongs to.

The candidate cells for each transcript are determined by a spatial receptive field constructed around each cell boundary of the input segmentation (by default the platform's cell segmentation, derived for example from membrane staining). For a boundary with area A, the effective radius is computed as $r = \sqrt{A/\pi}$, and the polygon is expanded by $r \times \gamma$, where $\gamma = 0.05$ by default. Scaling the buffer by the effective radius rather than a fixed distance is biologically motivated: larger cells have proportionally more cytoplasm, and cytoplasmic transcripts are distributed across a correspondingly larger physical area around the nucleus. Proportional expansion ensures that the receptive field captures a consistent fraction of each cell's transcriptome regardless of cell size. The default value $\gamma = 0.05$ adds a narrow margin around the cell outline, so that transcripts lying just outside it can still be considered, and represents a balanced operating point between sensitivity and specificity; larger values favour sensitivity and smaller values specificity (Section 1.7). For a typical cell of area 200 µm^2 ($r \approx 8$ µm), this corresponds to a buffer of approximately 0.4 µm. Transcripts whose coordinates fall within any expanded polygon are connected to the corresponding cell node by a directed edge, forming the candidate edge set $\mathcal{E}_{pred}$. Containment is evaluated with a GPU spatial join, and a transcript that lies strictly inside several buffered polygons becomes a candidate for each of them.

Training and prediction thus use different spatial support, controlled by two settings. The training-graph mode (default `nucleus`) determines which overlaps provide positive supervision (Section 1.2.4), and the prediction-graph mode (default `cell`) determines which staining-derived polygons define the candidate field. Training therefore remains anchored on the most reliable nuclear overlaps, while inference can recover cytoplasmic transcripts from the full cell area. In the alternative `nucleus` prediction mode, the candidate field is obtained by buffering each nuclear polygon by a multiple of its equivalent-circle radius ($\gamma = 0.5$, 1 or 2). This is a harder assignment task, because the cytoplasmic extent of each cell must be inferred from molecular context alone rather than bounded by a cell outline, and it mirrors assays without cell-boundary staining.

The width of the receptive field governs a tradeoff between contamination removal and transcript recovery. A narrow receptive field restricts each transcript to a small pool of candidate cells, making it more likely that transcriptionally inconsistent transcripts find no strong match and are left unassigned — maximizing decontamination at the cost of lower cytoplasmic transcript capture. A wider field extends the candidate pool, recovering more cytoplasmic transcripts but increasing the probability that a contaminating transcript finds a cell whose embedding it superficially matches. At the default setting, Segger achieves the best of both worlds: contamination is substantially lower than even conservative nuclear segmentation, while the fraction of transcripts assigned approaches that of cell boundary segmentation methods that explicitly model the cytoplasm. The right value of γ depends on the downstream analytical goal and is ultimately a biological judgment rather than a computational one; guidelines for setting it are discussed in Section 1.7.

For each transcript i, the cosine similarity s_{ij} is computed for all candidate cells $j \in \mathcal{E}_{pred}$, and the transcript is assigned to the most similar cell:

$$j^*(i) = \underset{j:\,(i,j)\in\mathcal{E}_{pred}}{\arg\max}\; s_{ij}. \tag{12}$$

The similarity score resolves competition among candidates by asking which cell the transcript's embedding is most consistent with transcriptionally — it is the mechanism by which contaminating transcripts are cleared and cytoplasmic transcripts are recovered within the same unified framework. No similarity threshold is applied at this step: a low score within a tight receptive field often reflects genuine biological ambiguity rather than a wrong assignment, and hard thresholding would silently discard real signal. The raw similarity score is stored alongside every assignment; per-gene adaptive thresholding (Section 1.5.2) then determines which assignments are retained, while low-confidence assignments remain available, flagged, for downstream inspection. Transcripts with no candidate cells in $\mathcal{E}_{pred}$ remain unassigned (cell ID = `null`). In the implementation, the argmax is computed for all transcripts of a tile in a single vectorized scatter-maximum over the candidate edges.

At inference, the encoder uses the existing transcript–transcript neighbourhood edges and the input transcript–cell overlap edges for message passing. The larger candidate transcript–cell edge set is used only to score and rank possible assignments after embedding, not to introduce an additional encoder relation.

Inference is performed on overlapping tiles, consistent with Section 1.4.1. Each tile is expanded outward by the same 20 µm margin used during training, ensuring that transcript and cell nodes near tile boundaries receive their full spatial neighborhood context and that assignments near tile boundaries are no less reliable than those in tile interiors. Only transcripts within the unexpanded inner tile boundary are output as final predictions, preventing duplicate assignments at tile boundaries. A tile-level mask marks the transcripts inside the inner bounds, so that a transcript appearing in several overlapping context margins is emitted by exactly one tile.

1.5.2 Per-gene similarity thresholding

The similarity score produced at assignment is not merely a ranking device — it carries biological information about how confidently each transcript belongs to its assigned cell. Per-gene adaptive thresholding translates

this continuous score into a principled decision on which assignments are retained, adding a second layer of transcriptional quality control on top of the spatial and similarity-based filtering already performed during assignment (Section 1.5.1). The two layers are complementary: assignment clears transcripts that find no strong match within their receptive field, while per-gene thresholding flags transcripts that were assigned but whose similarity score falls below the natural separation between confidently and ambiguously assigned transcripts for their gene.

A single global threshold would be inappropriate because similarity scores vary systematically across genes for two distinct reasons. First, genes differ in their spatial expression patterns: highly restricted marker genes expressed in dense, coherent cell populations — such as lineage-defining transcription factors or tissue-specific structural genes — yield consistently high assignment similarities because their transcripts reliably co-occur with cells whose embeddings reflect that gene program. Diffusely expressed or lowly expressed genes yield lower and more variable scores because their signal is distributed across many cell types and their contribution to any individual cell's embedding is weaker. Second, genes differ in their expression distributions — in dynamic range, modality, and the degree to which expression is concentrated in specific cell states. A global threshold would systematically over-retain diffusely expressed genes and over-discard spatially restricted ones, introducing gene-specific biases into the final assignment that would distort downstream analyses of cell type composition and gene expression heterogeneity. Per-gene adaptive thresholding respects both sources of variation, treating each gene's similarity distribution on its own terms.

For each gene, the similarity scores of all assigned transcripts are collected and, if necessary, subsampled to ≤ 10 million values with a fixed random seed for computational tractability while retaining a representative sample for stable threshold estimation. The score distribution for a well-behaved gene is expected to be roughly bimodal: a population of confidently assigned transcripts with high scores, and a tail of ambiguously assigned transcripts with lower scores reflecting uncertain or contested assignments. Two thresholds are computed automatically using scikit-image v0.26 to locate the natural boundary between these populations: Li's iterative minimum cross-entropy threshold [11] (limited to 250 iterations), which minimizes cross-entropy between the thresholded foreground and background distributions and is robust when bimodality is weak or the two populations are unequal in size, and Yen's maximum-correlation threshold [12], which maximizes correlation between the thresholded result and the original distribution and tends to select a slightly higher cutoff. The minimum of the two is taken as the final threshold for that gene, selecting the more lenient value. In the rare case that Li's iteration does not converge within its step limit, Segger v0.2.0 assigns the gene a fallback threshold equal to the median among all successfully estimated gene thresholds and records `converged=false` in the output file; by default these genes are filtered out for downstream analyses. Leniency is the correct default at this stage: downstream analyses operate on full cell profiles with access to co-expression context, cell neighborhood information, and biological priors that are unavailable at the level of individual transcript assignments. Irreversible discarding of ambiguous transcripts here forecloses interpretations that may be recoverable later; retaining their assignment scores and thresholds in the transcript-level output preserves the option to revisit the cutoff.

Transcripts whose similarity falls below their gene's threshold remain in the transcript-level output, although they are excluded from the cell-by-gene matrix. Formally, the assignment of transcript t_i to its highest-scoring candidate cell $j^*(i)$ is retained when $K(t_i) = 1$, where

$$K(t_i) = \mathbf{1}\{j^*(i) \text{ exists and } s_{ij^*(i)} \geq \tau_{g_i}\}, \tag{13}$$

τ_g is the threshold of gene g; when the Li threshold estimate does not converge, Segger v0.2.0 uses the median of the successfully estimated gene thresholds as a fallback, and marks that gene with `converged=false`. Transcripts without a candidate cell have $K(t_i) = 0$. The Segger segmentation is the set $\{t_i \in \mathcal{V}_{tx} : K(t_i) = 1\}$, from which the cell-by-gene matrix is built. The transcript-level output preserves the raw assignment and similarity score even below the threshold; v0.2.0 does not write a `filtered` column. Both the assignment and the per-gene threshold are stored alongside the raw similarity score, giving the downstream analyst full visibility into the confidence of every assignment and the freedom to apply whatever filtering criteria their biological question demands. Segger's output retains the scores, thresholds and gene-level convergence indicators needed for downstream review of transcript assignments.

1.6 Segger software

Segger is open-source under the MIT license and distributed on GitHub.[1] The analyses reported here used Segger v0.2.0 with Python 3.11, PyTorch 2.5, PyTorch Geometric 2.7 [4] and PyTorch Lightning 2.6 [13]; v0.2.0 is a tagged public release (21 August 2026) on GitHub. RAPIDS libraries (cuDF, cuML, cuGraph and cuSpatial, v24.10) and CuPy provide GPU dataframe, clustering, embedding and spatial-join operations. CUDA-dependent packages must be built against a compatible CUDA version; the reference environment uses CUDA 12.1 and is installed from a pixi lockfile (environment `cuda121`). All analyses trained and predicted on a single GPU in 32-bit

[1] https://github.com/dpeerlab/segger

precision. For reproducibility, we recommend using the v0.3.1 release, which is available at https://github.com/dpeerlab/segger/releases/tag/v0.3.1. It is algorithmically identical to v0.2.0, but simplifies installation with pixi, adds export to SpatialData, speeds up tile access during training and has improved documentation. Segger does not set a global random seed, so repeated runs on the same input differ slightly.

The Atera spatial transcriptomics platform, released in 2026, makes segmentation computationally more challenging. Gene panels increase from roughly 5k to more than 18k genes, and datasets can exceed 1 billion transcripts. The resulting graphs contain more edges than 32-bit integer indices can handle, which breaks several PyTorch and RAPIDS operations. To support this increase in data volume, we replaced cuSpatial with `cuspa`, a library developed by NVIDIA, for the point-in-polygon join, which is the most computationally demanding spatial operation. We also replaced the cuSpatial quadtree, which we found to be unstable and is limited to 32-bit indices, with `fastquadtree`, a CPU implementation that we apply to randomly downsampled data. Downsampling to 50 million transcripts and applying `fastquadtree` yielded quadtrees that were 98% identical to those built by the cuSpatial quadtree on the full data. Several further changes reduce the memory footprint and allow entire Atera-scale datasets to be processed on a single A100 GPU. This version requires CUDA 13 and Python 3.13, and was used only for Figure 2. It is available at https://github.com/dpeerlab/segger/releases/tag/v0.4.0-manuscript.

1.7 Parameter selection guidelines

Segger provides a common set of default parameters for use across tissue types and imaging platforms, with a higher segmentation-loss weight used for the denser Atera dataset. User adjustments primarily concern two considerations: selecting the balance between transcript recovery and assignment specificity appropriate for the downstream analysis, and sizing graph partitions and batches to fit available computational resources.

Segger provides three controls over the balance between transcript recovery and assignment specificity: the spatial receptive field for candidate transcript–cell assignments, the similarity thresholds for retaining assignments, and the relative weights of the training losses. We recommend adjusting the receptive field to select the desired balance. Users can also raise similarity thresholds to filter a completed segmentation or change the loss weights before training.

The receptive field determines how far beyond each staining-derived outline transcripts are considered for assignment. Its expansion factor, γ, specifies a buffer proportional to the outline's equivalent-circle radius (Section 1.5.1). For cell masks, we recommend starting with $\gamma = 0.05$; for nucleus-only inputs, the tested range was $\gamma = 0.5$–2. Larger fields generally favor transcript recovery, whereas smaller fields favor specificity. This setting can be changed after training: increasing the receptive field and rerunning inference allows additional transcripts to be considered for assignment without retraining the model.

Gene-specific similarity thresholds are estimated automatically (Section 1.5.2). For greater specificity after a completed run, users can raise these thresholds and regenerate the cell-by-gene matrix from the stored assignments and similarity scores. This removes low-scoring assignments without retraining or rerunning inference. Threshold adjustment cannot recover transcripts outside the candidate field used for that run; broader spatial recovery requires expanding the receptive field and rerunning inference.

Loss weights balance adherence to the initial transcript–cell assignments against consistency with cell-expression and gene-co-expression structure. Unlike the inference-time controls above, changing these weights requires retraining and is therefore recommended only after considering the others. By default, the cell and transcript losses have constant unnormalized weights of 1, while the segmentation-loss weight increases from 0 to 0.5 during training (Section 1.3.3). For the much denser Atera dataset, the segmentation loss had not converged within the standard training period; we increased its final weight to 1.0 while retaining the other weights and training duration.

The default transcript graph connects each transcript to at most four neighboring transcripts within 5 µm, together with a self-loop. We recommend retaining these settings; increasing the number of graph edges increases computation and memory requirements. Tile capacity (50,000 nodes) and the batch edge budget (one million edges) can be reduced for GPUs with less memory, while tile margins should preserve approximately one cell diameter of spatial context. For the much denser Atera dataset only, we increased the batch edge budget to four million edges to provide sufficient examples for contrastive learning within each batch.

2 Results

2.1 Segger scales to billion-transcript whole-transcriptome iST

Imaging-based spatial transcriptomics is moving from targeted gene panels toward whole-transcriptome profiling, which greatly increases both the number of detected molecules and their spatial density. To test whether Segger is effective at this level of data complexity, we applied it to a publicly available Atera section of cervical squamous-cell carcinoma containing 1.04 billion transcripts across 18,028 genes. With approximately 23.1 transcripts per

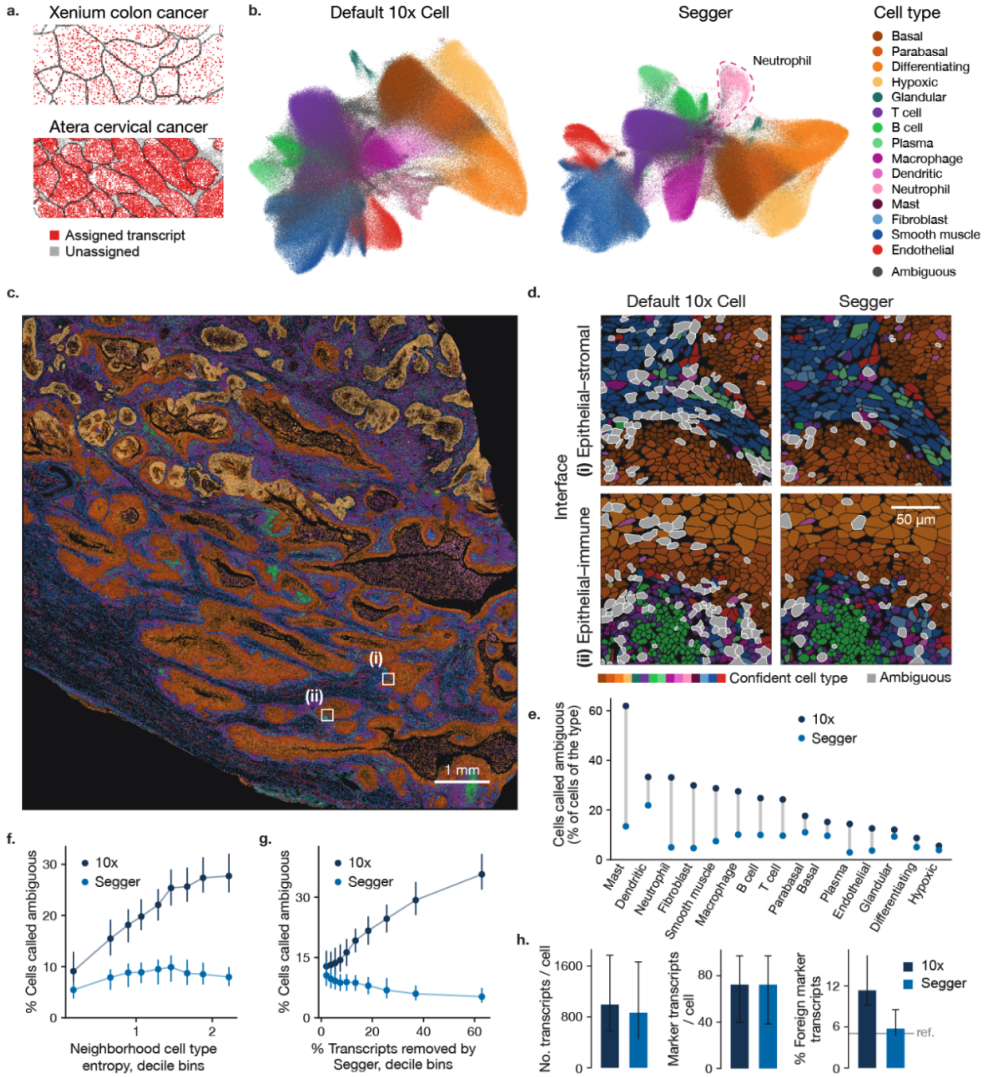
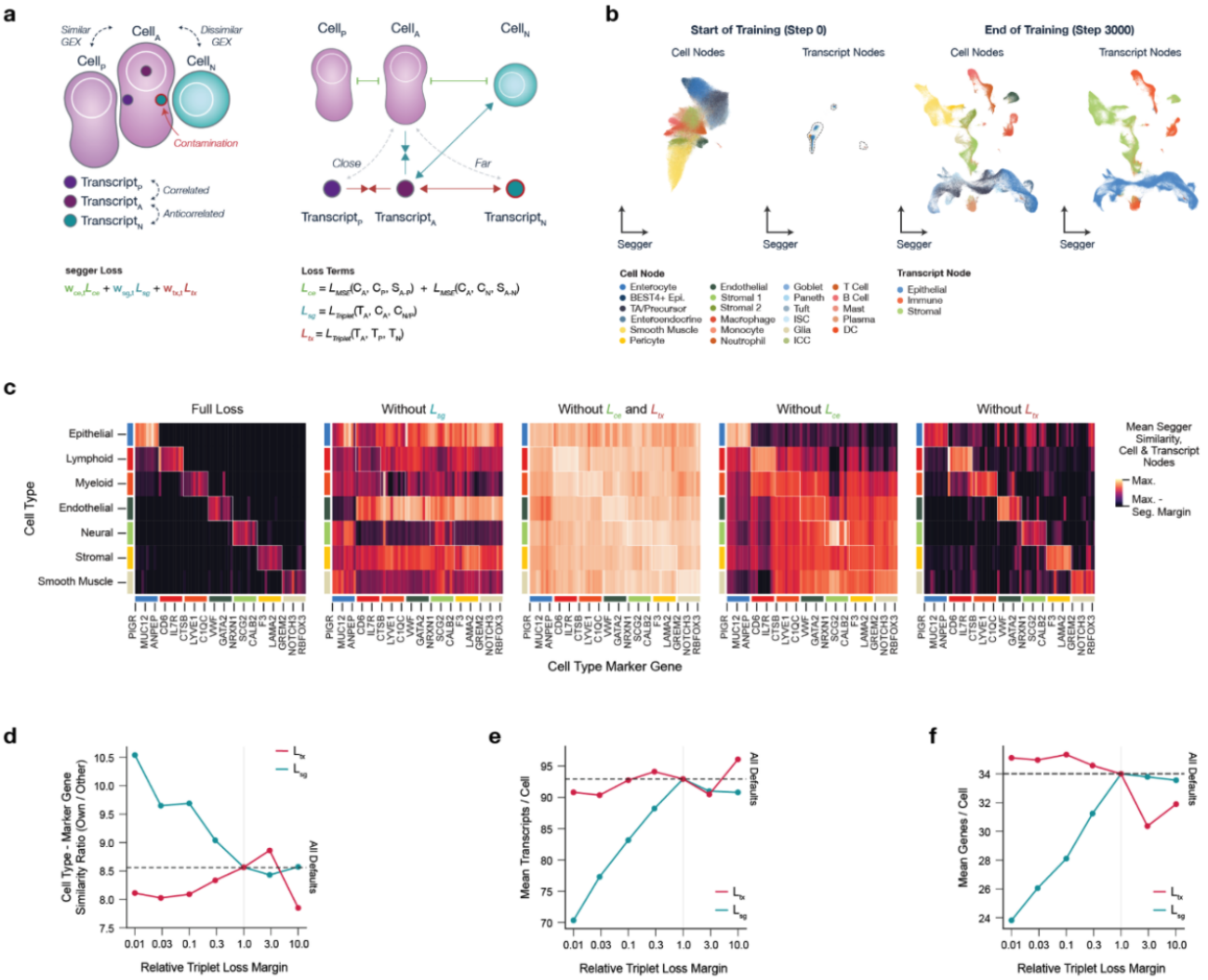


Figure 2 | Segger scales to whole-transcriptome iST and reduces contamination relative to 10x Cell. Cell types were assigned by scoring cluster-mean profiles against a curated cervical marker panel with ScType (Methods). (a) Representative FOVs highlight the difference in transcript density between the Xenium colorectal cancer and Atera cervical cancer datasets (average 120 and 2,100 transcripts/cell, respectively). (b) UMAPs of all cells of the 10x Cell (714,210 cells) and Segger (670,998 cells) segmentations, coloured by cell type, with ambiguous calls as large dark grey dots. Among paired cells, 20.1% of 10x calls and 8.2% of Segger calls are ambiguous. (c) Segmentation of the full cervical squamous-cell carcinoma tissue section, with each paired cell at its 10x centroid coloured by its Segger cell type. Boxes mark the fields of view in (d). (d) 200-µm fields of view at the epithelial–stromal interface (i) and an aggregate of B cells meeting tumour epithelium (ii), segmented by 10x (cells drawn with their boundaries) or Segger (cells drawn as the convex hull of the transcripts they keep). (e) Percentage of ambiguous cells per cell type. (f) Percentage of ambiguous cells as a function of the mean Shannon entropy, binned by decile, of the cell types of a cell's 15 nearest neighbours, counting only neighbours that both segmentations call confidently and identically. Error bars, interquartile range of each decile's value across 500-µm squares of the tissue with at least 50 cells. Spread is shown across squares to measure how the percentage varies across the tissue. (g) Mean percentage of ambiguous cells against share of a 10x Cell's transcripts that Segger does not keep. Deciles and error bars as in (f). The more Segger removes, the more ambiguous cells 10x Cell calls (13% to 36%) and the fewer ambiguous cells Segger calls (11% to 5%). (h) Left, median transcripts per cell, for cells matched between segmentation tools (10x 993 [579–1,771]; Segger 865 [449–1,666]). Center, mean epithelial, lymphoid, myeloid and stromal lineage-marker transcripts per cell of the same lineage (10x 72.4; Segger 72.3). Right, percentage of a cell's lineage-marker transcripts from other lineages, pooled over matched cells; each segmentation scored by its own annotation. Error bars, interquartile range across 500-µm squares with at least 50 cells. Dashed line, same measure in reference cells (5.0%).

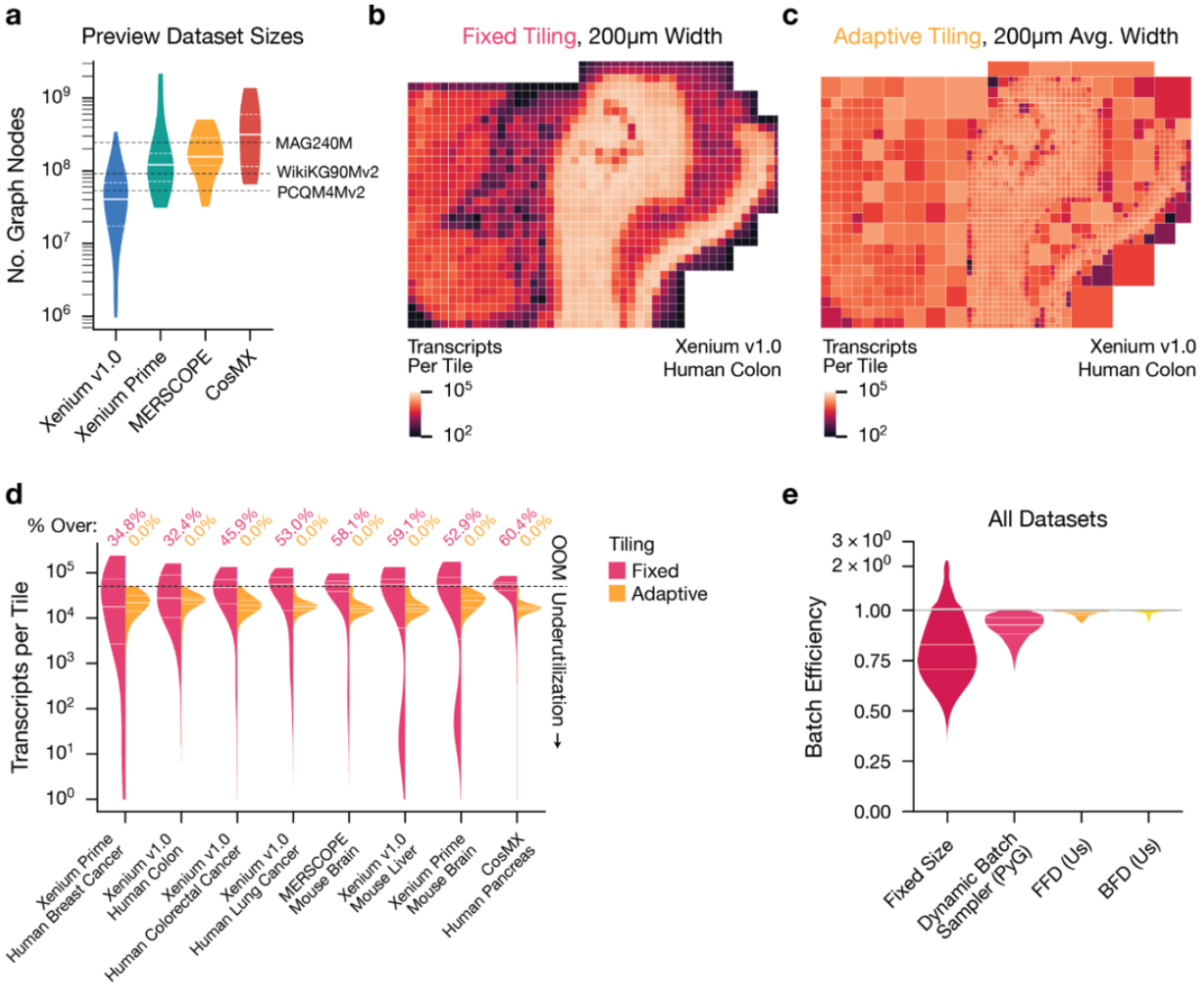
μm^2, compared with 1.2 transcripts per μm^2 in a representative Xenium dataset, Atera represents a substantial change in scale from targeted-panel iST (**Fig. 2a**). At this density, Segger operates on a graph containing billions of edges, requiring substantial re-engineering. We therefore developed a new GPU-native implementation of Segger based on NVIDIA cuspa (1.6), including scalable spatial operations such as point-in-polygon queries. The model formulation, learning objectives and transcript-to-cell assignment framework are unchanged, allowing Segger to segment a complete Atera section at whole-transcriptome scale.

At this scale, Segger produced substantially cleaner cell-type separation than 10x Cell (**Fig. 2b**). Among paired cells, 20.1% of cell-type assignments by 10x Cell were ambiguous, compared with only 8.2% by Segger. Spatially, these ambiguous 10x Cell cells were enriched at interfaces between different lineages, including epithelial–stromal and epithelial–immune boundaries (**Fig. 2c,d**), while Segger reduced ambiguity across diverse cell types (**Fig. 2e**). At the high molecular density of Atera, geometric boundaries alone provide limited information about the origin of transcripts from closely apposed cells. Consistent with this, ambiguity in the 10x Cell segmentation increased sharply as the cell-type diversity of the local neighborhood increased, whereas Segger remained substantially less ambiguous (**Fig. 2f**). Segger can resolve these difficult assignments because its link-prediction framework constrains transcripts to spatially plausible cells, while its biology-aware latent representation evaluates their compatibility with cell state and local molecular context. It therefore provides the greatest improvement in cell-type resolution where distinct lineages are spatially intermixed, precisely where transcript-to-cell assignment is most difficult.

Although Segger retained fewer transcripts per cell than 10x Cell, these transcripts provided a more faithful representation of cellular identity (**Fig. 2g,h**). We used a matched 10x Chromium Flex dataset to define highly specific markers of each major lineage, providing an independent measure of both retained lineage signal and cross-lineage contamination (Methods). Segger retained essentially the same number of own-lineage marker transcripts as 10x Cell (72.3 versus 72.4 per cell), while reducing the fraction of foreign-lineage marker transcripts by approximately half, approaching the level observed in the dissociated Flex reference (**Fig. 2h**). Consistent with this selective removal, cells from which Segger removed progressively more 10x Cell-assigned transcripts were increasingly likely to have ambiguous 10x Cell cell-type calls and less likely to be ambiguous after Segger assignment (**Fig. 2g**). Thus, even at whole-transcriptome scale, Segger preferentially removes cross-lineage contamination while preserving the lineage-defining signal of the cell.



Extended Data Fig. 1 | A composite metric-learning objective shapes the joint transcript–cell embedding. (a) Left, schematic of the training signal. Given an anchor cell (CellA), a cell with similar (CellP) or dissimilar (CellN) expression profile should embed nearby or far from it, respectively; similarly, transcripts of correlated genes attract and those of anticorrelated genes repel. A transcript that is initially misassigned to a neighboring cell (red) is therefore drawn toward the cell whose profile it matches rather than the cell it happens to lie within. Right, the three components of the objective and their scheduled weights. The cell loss ($\mathcal{L}_{bd}$) regresses the cosine similarity of a pair of cell embeddings onto the target similarity S of their reference clusters, for one sampled positive and one sampled negative. The segmentation loss ($\mathcal{L}_{seg}$) is a triplet margin loss anchored on a transcript, taking its assigned cell as the positive and a randomly drawn other cell as the negative. The transcript loss ($\mathcal{L}_{tx}$) is a triplet margin loss over transcripts, with positives and negatives sampled in proportion to gene-cluster similarity and dissimilarity. Weights are ramped over training on a cosine schedule and renormalized to sum to one at every step. (b) Joint embedding of a fixed sample of cells and their assigned transcripts from the healthy human colon Xenium v1.0 dataset, at initialization (step 0) and after convergence (step 3000). Cell nodes are colored by cell type and transcript nodes by the compartment of the cell they are assigned to. Checkpoints were embedded with AlignedUMAP, linking each node to itself across steps, so that coordinates are comparable between the two snapshots. Epi., epithelial; TA, transit-amplifying; ISC, intestinal stem cell; ICC, interstitial cell of Cajal; DC, dendritic cell. (c) Ablation of each loss component. Color, mean Segger similarity between transcripts of a given marker gene and the cells they were assigned to, restricted to cells of a given type; columns are the five most differentially expressed genes of each of seven cell type groups (rows), ordered by hierarchical clustering of the full-objective matrix and held fixed across panels. Left, the complete objective; remaining panels, models retrained with the segmentation loss, both the cell and transcript losses, the cell loss alone, or the transcript loss alone removed. The color scale spans one segmentation-loss margin below the maximum similarity, and dotted boxes mark each group's own markers. Removing the cell and transcript losses collapses the contrast between a cell type's own markers and all other genes, showing that the segmentation loss alone does not make transcript assignments discriminative. (d) Cell-type specificity across a triplet-margin sweep. Ratio of the mean Segger similarity between cells of a type and their own marker genes to the mean similarity with the marker genes of all other types, that is, the on- versus off-diagonal contrast of the matrices in (c). The segmentation margin ($\mathcal{L}_{seg}$) and the transcript margin ($\mathcal{L}_{tx}$) were each scaled relative to their default while the other was held fixed. (e) Mean transcripts assigned per segmented cell across the same sweep. (f) Mean genes detected per segmented cell across the same sweep. In (d–f), the vertical line marks the default relative margin and the dashed horizontal line the default Segger configuration ($m_{tx} = 0.2$, $m_{seg} = 0.4$); each point is one retrained model.



Extended Data Fig. 2 | Adaptive tiling and bin-packed batching keep iST graphs within a fixed memory budget. (a) Distribution of graph sizes across 108 publicly released ("preview") iST datasets, grouped by platform; graph nodes are the sum of detected transcripts and segmented cells reported by each vendor's onboard analysis (Xenium v1.0, n = 63 sections; Xenium Prime, n = 11; MERSCOPE, n = 26; CosMx, n = 8). Dashed lines, node counts of the largest Open Graph Benchmark Large-Scale Challenge graph-learning benchmarks. Violin plot solid line, median; dashed lines, quartiles. (b) Transcripts per tile under fixed square tiling (200-µm side length) of the healthy human colon Xenium v1.0 dataset. (c) Adaptive quadtree tiling of the same dataset, with leaf capacity chosen so that the mean tile width matches (b). Tiles shrink in transcript-dense regions and expand in sparse ones, equalizing occupancy rather than area. (d) Transcripts per tile under different tiling regimes for eight datasets spanning three platforms. Adaptive tiling used a leaf capacity of 50,000 transcripts; fixed tiling used a per-dataset side length chosen so that the average tissue-covered tile holds the same 50,000 transcripts. Dashed line, 50,000-transcript budget; percentages, occupied tiles exceeding the budget. Tiles above budget risk out-of-memory (OOM) failure, while the long lower tail of the fixed distributions represents underutilized compute. (e) Batch efficiency (transcripts assigned to a batch divided by batch capacity of 150,000 transcripts) for four strategies applied to the same adaptive tiles and pooled across all eight datasets: fixed number of tiles per batch, greedy sequential accumulation used by PyTorch Geometric DynamicBatchSampler, and first-fit-decreasing (FFD) and best-fit-decreasing (BFD) bin packing used by Segger. Grey line, capacity; efficiencies above 1 (log-scaled upper panel) overflow the budget and would OOM; efficiencies below 1 waste allocated memory.

Code availability

Segger is available as open-source software under the MIT license at https://github.com/dpeerlab/segger.

References

[1] G. Corso, H. Stark, S. Jegelka, T. Jaakkola, and R. Barzilay. Graph neural networks. *Nature Reviews Methods Primers*, 4(1):17, 2024. doi: 10.1038/s43586-024-00294-7.

[2] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=F72ximsx7C1.

[3] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph Attention Networks. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=rJXMpikCZ.

[4] M. Fey and J. E. Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019. URL https://arxiv.org/abs/1903.02428.

[5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[6] Weihua Hu, Matthias Fey, Hongyu Ren, Maho Nakata, Yuxiao Dong, and Jure Leskovec. OGB-LSC: A large-scale challenge for machine learning on graphs. *arXiv preprint arXiv:2103.09430*, 2021.

[7] William L. Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, pages 1024–1034, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7ebea9-Abstract.html.

[8] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. GraphSAINT: Graph sampling based inductive learning method. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=BJe8pkHFwS.

[9] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 257–266, 2019. doi: 10.1145/3292500.3330925.

[10] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1412.6980.

[11] C. H. Li and C. K. Lee. Minimum cross entropy thresholding. *Pattern Recognition*, 26(4):617–625, 1993. doi: 10.1016/0031-3203(93)90115-D.

[12] Jui-Cheng Yen, Fu-Juay Chang, and Shyang Chang. A new criterion for automatic multilevel thresholding. *IEEE Transactions on Image Processing*, 4(3):370–378, 1995. doi: 10.1109/83.366472.

[13] William Falcon et al. Pytorch lightning. https://github.com/PyTorchLightning/pytorch-lightning, 2020.